R1.01 : Initiation au développement et SAÉ 1.01 : Implémentation d’un besoin client

R1.01 : Initiation au développement et SAÉ 1.01 : Implémentation d’un besoin client

Date de publication 2025

Table des matières

1. R1.01 : Initiation au développement 1
    1. Programme National (PN) 1
        1.1. Compétences ciblées 1
        1.2. Descriptif 1
        1.3. Savoirs de référence étudiés 1
        1.4. Planning 1
        1.5. Présentation d'un langage de programmation : le langage C 2
            1.5.1. Cours 2
            1.5.2. Mise en pratique 11
        1.6. Programmation en go 17
            1.6.1. Apprentissage du GO 17
            1.6.2. Travaux pratiques 52
2. SAÉ 1.01 : Implémentation d'un besoin client 55
    1. Évaluation et dates 56
    2. Principe général d'un système de particules 57
        2.1. Particule 57
        2.2. Système de particules 57
    3. Organisation des sources du projet 57
        3.1. Dossier principal 57
            3.1.1. Les fichiers go à la racine 57
            3.1.2. Fichier de configuration 58
        3.2. Dossier assets 58
        3.3. Dossier config 58
        3.4. Dossier particles 58
    4. Premier travail : système de particules de base 59
        4.1. Fonction NewSystem() 59
        4.2. Fonction Update() : vitesse des particules 59
        4.3. Fonction Update() : génération de particules 60
    5. Deuxième travail : extensions 60
        5.1. Gravité (Simple, pas de difficulté algorithmique) 60
        5.2. Extérieur de l'écran (Simple, pas de difficulté algorithmique) 61
        5.3. Durée de vie (Simple, pas de difficulté algorithmique) 61
        5.4. Variations de couleur, d'échelle, de rotation, de transparence (Simple, pas de difficulté algorithmique, rendu visuel) 61
        5.5. forme du générateur (Simple, pas de difficulté algorithmique, rendu visuel) 61
        5.6. Générateur mobile (Difficulté algorithmique moyenne, rendu visuel) 62
        5.7. Modification dynamique du système de particules (Ergonomie, pas de difficulté algorithmique) 62
        5.8. Optimisation de la mémoire (Purement algorithmique, difficulté moyenne à difficile) 62
        5.9. Optimisation de la mémoire ++ (Purement algorithmique, difficile) 62
        5.10. Collisions (Purement algorithmique, difficile) 63

Liste des illustrations

2.1. Particules exemple 1 55
2.2. Particules exemple 2 56

Liste des tableaux

1.1. Planning prévisionnel 1

Liste des exemples

1.1. hello.go 17
1.2. vars.go 18
1.3. vars2.go 19
1.4. calcul.go 20
1.5. tabs.go 21
1.6. tabs2.go 21
1.7. tabs3.go 21
1.8. tabs4.go 22
1.9. for.go 22
1.10. for2.go 23
1.11. for3.go 24
1.12. if.go 25
1.13. if2.go 25
1.14. if3.go 26
1.15. switch.go 27
1.16. func.go 27
1.17. func2.go 28
1.18. func3.go 29
1.19. struct.go 29
1.20. struct2.go 29
1.21. struct3.go 30
1.22. valeur-reference.go 31
1.23. pointeurs.go 31
1.24. pointeur2.go 32
1.25. valeur-reference2.go 33
1.26. methode.go 33
1.27. methode2.go 34
1.28. array.go 35
1.29. slice.go 35
1.30. slice2.go 36
1.31. slice3.go 36
1.32. slice4.go 37
1.33. slide5.go 37
1.34. slice6.go 37
1.35. slide7.go 38
1.36. slice8.go 39
1.37. slice9.go 39
1.38. tabtab.go 40
1.39. args.go 41
1.40. flag.go 42
1.41. create.go 43
1.42. write.go 43
1.43. readfile.go 44
1.44. fscanln.go 45
1.45. bufio.go 47
1.46. interface.go 49
1.47. anonyme.go 50
1.48. tri.go 50
1.49. applique.go 51
1.50. tri2.go 52

Chapitre 1. R1.01 : Initiation au développement

1. Programme National (PN)

1.1. Compétences ciblées

- Développer — c’c'est-à-dire concevoir, coder, tester et intégrer — une solution informatique pour un client.
- Proposer des applications informatiques optimisées en fonction de critères spécifiques : temps d’exécution, précision, consommation de ressources.

1.2. Descriptif

L’objectif de cette ressource est l’initiation au développement. Cette ressource est nécessaire pour la réalisation d’un développement d’application et l’optimisation des applications informatiques.

1.3. Savoirs de référence étudiés

- Algorithmes fondamentaux (structures simples, recherche d’un élément, parcours, tri...)
- Algorithmes sur les structures de données (itératifs et/ou récursifs)
- Manipulation de listes, tableaux, collections dynamiques, statiques (accès direct ou séquentiels), piles, files, structures
- Types abstraits de données simples : première approche de l’encapsulation
- Notions de modularité – Premières notions de qualité (par ex. : nommage, assertions, documentation, sûreté de fonctionnement, jeu d’essais, performance...)
- Lecture/écriture de fichiers

1.4. Planning

Tableau 1.1. Planning prévisionnel

Semaine	CM	TD	TP	Thématique
S36	1 Intro + C			Présentation
S37	1 Algorithme	3 (langage C)		Apprentissage d'un langage de programmation le C
S38	1 Recherche d'éléments	3 (langage C / partie 1 : variables, tableaux, boucles, conditionnelles, fonctions, structures)		FIn du C , Apprentissage du go.
S39	1 Récursivité	3 (partie 1 / partie 2 : pointeurs, tableaux, arguments)		Apprentissage du go.
S40	1 Tri	2 évaluation (C + go)	2 recherche	Recherche

S41	1 Structures de données	2 (partie 2 / partie 3)	2 recherche / récursivité	Récursivité
S42	1 Modules en go	2 partie 3 : fichiers	2 récursivité	
S45	1 Tester / gestion de version	2 évaluation	2 structures + pointeurs	Pointeurs / Structires
S46	1 Divers	2 Partie 4 : interfaces et fonctions "avancées"	2 structures + pointeurs	
S47		2 Début SAE	2 fichiers	Fichiers
S48	1 Présentation de la SAE	2 évaluation	2 fichiers / tri	Tri
S49		2 SAE	2 tri	
S50		2 SAE	2 SAE	SAE Rendu partie 1
S51		2 évaluation	2 SAE	
S2			3 SAE	
S3			2+1 SAE évaluation	Évaluation
S4			2 évaluation +1 évaluation par les pairs	Évaluation et par les pairs

1.5. Présentation d'un langage de programmation : le langage C

1.5.1. Cours

Le langage C est un langage de programmation impératif, généraliste, non objet et de "bas niveau", inventé au début des années 1970. Nous allons utiliser la version la plus contemporaine la C23 de 2024. Nous abordons ce langage en premier pour poser les bases de la programmation et comme un prérequis pour d'autres ressources.

1.5.1.1. La compilation

La compilation consiste à transformer un code source lisible par un humain en un fichier binaire exécutable par une machine. Le langage C est un langage compilé mais le processus est enrichi d'autres opérations pour augmenter la modularité :

Traitement par le préprocesseur

> Le préprocesseur C ou cpp assure une phase préliminaire de la traduction (compilation) des programmes informatiques écrits dans les langages de programmation C et C++. Comme préprocesseur, il permet principalement l'inclusion d'un segment de code source disponible dans un autre fichier (fichiers d'en-tête ou header), la substitution de chaînes de caractères (macro définition), ainsi que la compilation conditionnelle[1]. Nous utiliserons les directives #include (recopie de fichier), #define (définition de macro et expansion). D'autres existent comme #if, #else, #endif pour la compilation conditionnelle, #error pour les erreurs personnalisées, #pragma, #warning, ...

Compilation

> La compilation proprement dite traduit le fichier généré par le préprocesseur en assembleur, c'est-à-dire en une suite d'instructions du microprocesseur qui utilisent des mnémoniques rendant la lecture possible[2].

Assemblage

> cette opération transforme le code assembleur en un fichier binaire, c'est-à-dire en instructions directement compréhensibles par le processeur. Généralement, la compilation et l'assemblage se font dans la foulée, sauf

[1]Source wikipédia
[2]Source Anne Canteaut

si l'on spécifie explicitement que l'on veut le code assembleur. Le fichier produit par l'assemblage est appelé fichier objet.

Édition de liens

un programme est souvent séparé en plusieurs fichiers source, pour des raisons de clarté mais aussi parce qu'il fait généralement appel à des librairies de fonctions standard déjà écrites. Une fois chaque code source assemblé, il faut donc lier entre eux les différents fichiers objets. L'édition de liens produit alors un fichier dit exécutable.

Les différents types de fichiers utilisés lors de la compilation sont distingués par leur suffixe. Les fichiers source sont suffixés par .c, les fichiers prétraités par le préprocesseur par .i, les fichiers assembleur par .s, et les fichiers objet par .o. Les fichiers objets correspondant aux librairies pré-compilées ont pour suffixe .a.

Voici une petite illustration :

Le fichier source .c

Tout commence par un fichier source, construit avec un éditeur de texte seul ou dans un environnement de développement

```
//hello.c

//Inclusion de
//En-tête Standard d'Entrée/Sortie
#include <stdio.h>
//Définition d'une macro
#define USER "jub"

//La fonction main
int main(int argc, char *argv[]){
 //l'instruction d'appel à la fonction  printf de la bibliothéque stdio
 printf("Bonjour %s\n", USER);
 //fin du main et renvoie du code retour 0
 return 0;
}
```

Le fichier après processeur .i

GCC (GNU Compiler Collection) est l'outil que nous allons utiliser pour produire des exécutable, la documentation est accessible via man gcc.

La ligne de commande permet d'invoquer le préprocesseur et de stocker le résultat dans le fichier hello.i : gcc -E hello.c -o hello.i

On retrouve les inclusions des entêtes (types, signature des fonctions, ...) et les macro remplacées par leurs valeurs, les commentaires ont disparus :

```
typedef unsigned char __uint8_t;
typedef short __int16_t;
...
int printf(const char * restrict, ...)
...
int main(int argc, char *argv[]){

 printf("Bonjour %s\n", "jub");

 return 0;
}
```

Le fichiers après compilation .s

En utilisant gcc nous pouvons obtenir l'assembleur qui est dépendant du type de processeur : gcc -S hello.i.

```
.section __TEXT,__text,regular,pure_instructions
.build_version macos, 15, 0 sdk_version 15, 5
```

```
.globl _main ; -- Begin function main
.p2align 2 _main: ; @main
.cfi_startproc
; %bb.0:
sub sp, sp, #48
stp x29, x30, [sp, #32] ; 16-byte Folded Spill
add x29, sp, #32
.cfi_def_cfa w29, 16
.cfi_offset w30, -8
.cfi_offset w29, -16
mov w8, #0 ; =0x0
...
ret
.cfi_endproc
 ; -- End function
.section __TEXT,__cstring,cstring_literals
l_.str: ; @.str .asciz "Bonjour %s\n"
l_.str.1: ; @.str.1
.asciz "jub"
.subsections_via_symbols
```

Le fichier après compilation .o (objet)

En utilisant gcc : gcc -c hello.s nous obtenons un fichier compilé ce fichier n'est par encore exécutable car il n'est pas lié aux bibliothèques et fichiers inclus.

A ce stade, le fichier ne peut plus être affiché si ce n'est comme code binaire avec par exemple od -x hello.o, il est également possible de désassembler avec des commandes comme objdump

```
0000000      facf    feed    000c    0100    0000    0000    0001    0000
0000020      0004    0000    01b8    0000    2000    0000    0000    0000
0000040      0019    0000    0138    0000    0000    0000    0000    0000
0000060      0000    0000    0000    0000    0000    0000    0000    0000
0000100      0080    0000    0000    0000    01d8    0000    0000    0000
0000120      0080    0000    0000    0000    0007    0000    0007    0000
0000140      0003    0000    0000    0000    5f5f    6574    7478    0000
0000160      0000    0000    0000    0000    5f5f    4554    5458    0000
0000200      0000    0000    0000    0000    0000    0000    0000    0000
0000220      004c    0000    0000    0000    01d8    0000    0002    0000
...
```

Le fichier exécutable

En utilisant gcc : gcc -o hello hello.o

Il est évidement possible de réaliser l'ensemble des opérations en une seule commande et quelques options supplémentaires comme la version du langage et l'affichage des erreurs : gcc -Wall -std=c23 hello.c -o hello. Nous compliquerons plus tard le processus avec la compilation de plusieurs fichiers interdépendants.

1.5.1.2. La base du langage

Le langage C est comme tout langage défini par une syntaxe et une sémantique. La syntaxe est définie la forme du langage, par exemple les accolades {} définissent des blocs d'instruction et le point virgule ; termine une instruction. La sémantique définie le sens par exemple char c permet de déclarer c comme étant une variable de type char, int *f() une fonction retournant un pointeur vers un int et non un pointeur vers une fonction int *(*a)(int *). La syntaxe est facile, la sémantique peut se révéler plus complexe.

1.5.1.2.1. Les identificateurs

Les identificateurs permettent de nommer une variable, une fonction, un type défini par typedef, struct, union ou enum, une étiquette. Ils sont composé de lettres, de chiffres et de _. Ils ne peuvent commencer par un chiffre, ni être des mots clefs du langage.

1.5.1.2.2. Les commentaires

Les commentaires de bloc sont définis par /* commentaire */, les commentaires de ligne par // commentaire.

1.5.1.2.3. Les mots clef

Le C ANSI contient 32 mots clefs.

Les spécificateurs de stockage

auto, register, static, extern, typedef

Les spécificateurs de type

char, double, enum, float, int, long, short, signed, struct, union, unsigned, void

Les qualificateurs de type

const, volatile

Les instructions de contrôle

break, case, continue, default, do, else, for, goto, if, switch, while

Divers

return, sizeof

1.5.1.2.4. Les types prédéfinis

Le C est un langage fortement typé, un type est un ensemble de valeurs et des opérations sur ces dernière. Le type d'un objet définit la façon dont il est représenté en mémoire. La mémoire de l'ordinateur se décompose en une suite continue d'octets. Chaque octet de la mémoire est caractérisé par son adresse, qui est un entier. Deux octets contigus en mémoire ont des adresses qui diffèrent d'une unité. Quand une variable est définie, il lui est attribué une adresse. Cette variable correspondra à une zone mémoire dont la longueur (le nombre d'octets) est fixée par le type.

Le type caractère est un type entier et permet de stocker un caractère sur un octet, il peut être signé ou non signé :

```
char c1 = 'A';
unsigned char c2 = 'a';
```

Vous verrez dans une autre ressource comment les entiers, les flottants et les caractères sont représentés en mémoire. Par exemple un signed char est compris entre $[2^{-7}, 2^{7}[$, un unsigned entre $[0,2^{8}]$, 0xFF ou 255 représente 255 en non signé et -1 en signé, ...

Le type entier est int, il peut être précédé d'un attribut de précision (short ou long) et/ou d'un attribut de représentation (unsigned) :

```
short int a; //un entier de taille (longeur) comprise entre char et int
int b; //un entier
long int a; //un entier de taille supérieure à celle d'un int
unsigned short int; //un entier court non signé
```

Les flottants permettent de représenter les nombres à virgule (flotante), trois tailles sont possibles :

```
float a; //simple précision
double b; //double précision
long double c; //quadruple précision
```

1.5.1.2.5. Les constantes

Il existe 5 types de constantes :

entière

Une constante entière classique peut-être initialisée de 3 manières :

```
int a = 10; //décimale
int b = 011; //octale (base 8) commence par 0
int c = 0xA; //hexadécimal (base 16) commence par 0x
```

```
//seulement sur certains compilateurs
//int c=0b1010; en binaire (base 2) commence par 0b
```

flottant

Les constantes réelles sont représentées par la notation classique par mantisse et exposant. L'exposant est introduit par la lettre e ou E :

```
double a = -10.10E-4;
float b = -10.10E-4F; //Le nombre est suffixé par F
long c = -10.10E-4L; //Le nombre est suffixé par F
```

caractère

Les caractères, des entiers particuliers, sont notés entre simple quote ('), les caractères ', ?,\ doivent être échappés avec \ : \', \?, \\.

Il est également possible d'utiliser le code octal ou hexadécimal du caractère : \173, \x7b par exemple pour le z.

Enfin, certains caractères non imprimables disposent d'une notation plus simple : \n nouvelle ligne,\r retour chariot, \t tabulation horizontale, \v tabulation verticale, \b retour arrière, \f saut de page, \a signal d'alerte

Chaîne de caractères

Les chaînes de caractères sont notées entre ", ce sont des tableaux de caractères le \permet d'écrire une chaîne de caractères sur plusieurs lignes :

```
char chaine[] = "Un texte un \
peu long";
```

En mémoire, la fin d'une chaîne de caractère est délimitée par le caractère \0 :

```
char hello[6];
hello[0]='h';
hello[1]='e';
hello[2]='l';
hello[3]='l';
hello[4]='o';
hello[5]='\0';
```

Les énumérations

Une énumération permet de créer une liste de constantes entières :

```
#include <stdio.h>

int main(){
    enum naturelPair { ZERO, DEUX, QUATRE, SIX };

    enum naturelPair z = ZERO;
    enum naturelPair d = DEUX;

    printf("%d \n", (int) z);
    printf("%d \n", ZERO);
    printf("%d \n", (int) d);
    printf("%d \n", DEUX);

 return 0;
}
```

1.5.1.2.6. Les opérateurs

L'affectation = que nous avons déjà utilisé, a quelques particularités : une conversion implicite de type et prend la valeur de ce qui vient d'être affecté.

```
//affecation.c
#include <stdio.h>
```

```
//La fonction main
int main(){
    int a=2,b = 2.1;
    printf("%d %d \n",a,b);
    printf("%d \n",a=44);
 return 0;
}
```

Les opérateurs arithmétiques : + addition, - soustraction, * multiplication, / division, % reste de la division (modulo).

Les opérateurs relationnels : > strictement supérieur, >= supérieur ou égal, < strictement inférieur, <= inférieur ou égal, == égal, != différent.

Les opérateurs logiques booléen : && et logique, || ou logique, ! négation logique.

Les opérateurs logiques bit à bit à ne pas confondre avec les booléens : & et, | ou inclusif, ^ ou exclusif, ~ complément à 1, << décalage à gauche, >> décalage à droite.

Les opérateurs d'affectation composée +=, -=, *=, /=, %=, &=, ^=, |=, <<=, >>= ils correspondent à l'opération suivi de l'affectation

```
#include <assert.h>

//La fonction main
int main(){
    int a =2;
    a +=10;
    //Si a ne vaut pas 12, le programme s'arréte
    //un message d'erreur est affiché
    assert(a == 12);
    return 0;
}
```

Les opérateurs d'incrémentation ++ et de décrémentation --, sous forme suffixée la valeur retournée sera l'ancienne et sous la valeur préfixée sera la nouvelle.

```
//incrementation.c
#include <assert.h>

int main(){
    int a =2;
    assert(++a == 3);
    assert(a == 3);
    assert(a++ == 3);
    assert(a == 4);
    return 0;
}
```

L'opérateur virgule, permet de réaliser plusieurs instructions en une seule expression évalué de la gauche vers la droite.

```
int a, b;
b = ((a = 10), (a + 4));
assert(b==14);
```

L'opérateur conditionnel ternaire, sous la forme condition ? expression-1 : expression-2, l'évaluation de l'expression-1 est retournée si la condition est vraie et sinon l'expression-2 :

```
//ternaire.c
#include <assert.h>

int abs(int a) {
 //Si a>0 alors a sinon -a
 return a > 0 ? a : -a;
}

//La fonction main
```

```
int main(int argc, char *argv[]){
 assert(abs(2)==2);
 assert(abs(-2)==2);
 assert(abs(0)==0);
 return 0;
}
```

L'opérateur de conversion de type, simplement sous la forme (type) objet :

```
int a = (int) 2.0; //2
 float b = (float) 3/2; //1.5
```

L'opérateur adresse & et l'opérateur d'indirection *. & retourne l'adresse mémoire et * la valeur référencée par une adresse :

```
int a=10;
 printf("valeur %d\n", a); //10
 printf("adresse %p\n", &a); //0x16eee2eac
 printf("valeur referencee par l'adresse %d\n", *(&a)); //10
```

1.5.1.2.7. Les instructions de branchement conditionnel

Deux instructions : le branchement conditionnel et le branchement multiple.

Le branchement conditionnel sous la forme :

```
if (expression)
  instruction-1
[else
  instruction-2]
```

Le type booléen (vrai, faux) n'existe pas en C, 0 est faux, tout le reste est vrai. Si l'expression est évaluée, si elle est vrai alors l'instruction-1 et exécutée sinon l'instruction-2 est exécutée. La partie [else instruction-2] est optionnelle.

Des exemples permettant de tester la parité de la variable a :

```
//les accolade ne sont utiles que si il y a plus
//d'une instruction
int a=10;
 if (a%2==0)
  printf("a est pair\n");
 else
  printf("a est impair\n");
```

```
int a=10;
 char res[13] = "a est impair";

 if (a%2==0)
  //strcpy copie dans res, "a est pair"
  strcpy(res,"a est pair");

 printf("%s\n", res);
```

Le branchement multiple :

```
switch (expression)
  {case constante-1:
    liste d'instructions 1
    break;
  case constante-2:
    liste d'instructions 2
    break;
    ...
  case constante-n:
    liste d'instructions n
    break;
  default:
    liste d'instructions par def.
    break;
  }
```

Si la valeur de expression est égale à l'une des constantes, la liste d'instructions correspondant est exécutée. Sinon la liste d'instructions par def. correspondant à default est exécutée. L'instruction default est facultative.

```
int value=10;
 switch( value ) {
        case 1:
            printf( "One\n" );
            break;
        case 2:
            printf( "Two\n" );
            break;
        case 3:
            printf( "Three\n" );
            break;
        default:
            printf( "Other value\n" ); //on passe ici
    }
```

1.5.1.2.8. Les boucles

Trois boucles, la boucle while, la boucle do while, la boucle for.

La boule while while (expression) instruction. Tant que expression est vérifiée (i.e., non nulle), instruction est exécutée. Si expression est nulle au départ, instruction ne sera jamais exécutée.

```
//Les n premiers nombres
int n = 4;
 int i=0;
 while (i < n) {
  printf("%d\n",i);
  i++;
 }
 //0 1 2 3
```

A éviter d'utiliser les instructions de branchement non conditionnel : break, continue et goto à ne jamais utiliser.

Le break déjà vu avec le switch permet de quitter la boucle :

```
int n = 4;
 int i=0;
 while (i < n) {
  if (i==1) break;
  printf("%d\n",i);
  i++;
 }
 //0
```

Le continue, permet de passer à l'itération suivante :

```
int n = 4;
 int i=0;
 while (i < n) {
  if (i==1) continue;
  printf("%d\n",i);
  i++;
 }
 //boucle infinie
 //i n'est jamais augmenté
```

Dans la boucle do while, la conditionelle est à la fin, cette boucle impose donc au moins un passage.

```
int n = 4;
 int i=10;
 do  {

  printf("%d\n",i);
  i++;
 } while (i < n);
 //10
```

La boucle for est un cas particulier de la boucle while :

```
for (expr 1 ;expr 2 ;expr 3)
  instruction

//est equivalent à
expr 1;
 while (expr 2 )
   {instruction
    expr 3;
   }
```

```
for (int i=0; i<5;i++)
  printf("%d\n",i);
 //0 1 2 3 4
```

1.5.1.2.9. Les entrées/sorties de base

Nous avons déjà vu printf : printf("chaîne de contrôle ",expression-1, ..., expression-n);

Vous pouvez comme pour toutes les fonctions accéder à la documentation avec man 3 printf (q pour quitter, / pour rechercher /n pour le suivant et /p pour le précédent.

Pour lire nous allons utiliser scanf qui est formatée comme printf mais prend en second paramètre un pointeur (une variable qui stocke une adresse) :

```
char maChaine[5];
 //machaine est un pointer vers le premier
 //octet du tableau de caractères
 //scanf peut conduire à une dépassement
 //de mémoire si le texte saisi est trop long
 printf("Saisir un texte \n");
 int res = scanf("%s", maChaine);
 if (res == 0)
  printf("Lecture impossible");
 printf("Vous avez saisi %s  \n",maChaine);
```

1.5.1.2.10. La ligne de commande

Il est possible de passer des informations en ligne de commande à l'exécution d'un programme, c'est le rôle des paramètres argc et argv du main int main(int argc, char *argv[]).

argc donne le nombre d'éléments de la ligne de commande, et argv contient ces éléments sous la forme d'un tableau de chaînes de caractères. argv[0] contient le nom de la commande, argv[i], pour i allant de 1 à argc-1, le ième argument.

```
int main(int argc, char *argv[]){
 printf("Nombre de paramètres : %d\n",argc-1);
 for (int i=1; i<argc; i++)
  printf("Paramètre %d : %s \n",i,argv[i]);
}
```

Le programme précédent compilé en a.out et appelé comme suit ./a.out p1 p2 p3 contuira à l'affichage de

```
Nombre de paramètres : 3
Paramètre 1 : p1
Paramètre 2 : p2
Paramètre 3 : p3
```

1.5.1.2.11. Structure d'un programme

La structure de base d'un programme est

```
[directives au préprocesseur]
[fonctions secondaires]

main() //ou int main(int argc, char *argv[])
{déclarations de variables internes
```

```
  instructions
}
```

Nous avons vue les directives de préprocesseur : include et define. include existe sous deux formes

```
#include <systemFilename.h> //inclure un fichier de déclaration standard
#include "userFilename.h" //inclure un fichier de déclaration personnel
```

Les fonctions secondaires prennent la forme

```
type ma_fonction ( arguments )
{déclarations de variables internes
  instructions
}
```

Par exemple

```
int add(int a, int b) {
  int s;
  s = a+b;
  return s;
}
```

Une fonction qui ne renvoie rien est de type void :

```
void sayHello(){
  printf("Hello");
}
```

1.5.2. Mise en pratique

1.5.2.1. Partie 1 : introduction

1.5.2.1.1. Exercice 1.1

Écrire un programme qui lit un un entier au clavier et qui l'affiche ce dernier au carré :

1. Créer un fichier nommé exercice1.1.c avec un éditeur de texte ou un IDE
2. Partir de programme de base suivant :

   ```
   //exercice1.1
   #include <stdio.h>


   int main(int argc, char *argv[]){
    int unEntier;
    printf("Saisir un entier : ");
    //TODO
    printf("Son carrée est %d \n",unEntier);
   }
   ```

   et en utilisant la fonction scanf (man 3 scanf pour la documentation)
3. Compiler avec : gcc -Wall -std=c23 exercice1.1.c -o exercice1.1
4. Exécuter avec ./exercice1.1

1.5.2.1.2. Exercice 1.2

Écrire un programme qui lit un un flottant au clavier qui le transtype en un entier et qui l'affiche le carré entier de ce dernier. Par exemple 10,5 donnera 100.

1.5.2.1.3. Exercice 1.3

Nous allons maintenant créer une fonction carreEntier qui renvoie le carré entier et l'appeler depuis le programme principal. Des assertions sont fournies pour tester :

```
#include <assert.h>

int carreEntier(float n) {
 //TODO
}

int main(int argc, char *argv[]){
 assert(carreEntier(0)==0);
 assert(carreEntier(1)==1);
 assert(carreEntier(-1)==1);
 assert(carreEntier(-2.5)==4);
 assert(carreEntier(2.5)==4);

}
```

1.5.2.1.4. Exercice 1.4

Nous allons maintenant séparer le test de la définition de la fonction.

1. Le fichier exercice1.4.lib.c contiendra la fonction. Vous le compilerez avec l'option -c qui permet d'obtenir un fichier objet (.o).

2. Créer un fichier exercice1.4.lib.h (h comme header) qui contiendra la signature de la fonction :

   ```
   //exercice1.4.lib.h
   int carreEntier(float n);
   ```

3. Dans le fichier de test exercice1.4.test.c inclure le .h :

   ```
   //exercice1.4.test.c
   #include <assert.h>

   #include "exercice1.4.lib.h"

   int main(int argc, char *argv[]){
    assert(carreEntier(0)==0);
    assert(carreEntier(1)==1);
    assert(carreEntier(-1)==1);
    assert(carreEntier(-2.5)==4);
    assert(carreEntier(2.5)==4);

   }
   ```

   Compiler avec l'option -c pour obtenir les fichiers objets :gcc -Wall -std=c23 -c exercice1.4.lib.c et gcc -Wall -std=c23 -c exercice1.4.test.c

4. Nous pouvons enfin obtenir l'exécutable avec l'édition de lien, option -o : gcc -Wall -std=c23 exercice1.4.lib.o exercice1.4.test.o -o exercice1.4.

   Si vous deviez modifier le contenu de la fonction devriez-vous faire ?

1.5.2.1.5. Exercice 1.5

Il est possible d'automatiser le maintien des dépendances avec des outils de compilation, en C l'outil standard est le makefile. Le make file repose sur le principe des dépendances :

```
cible: dépendances
    commande
```

Voici un exemple de makefile que vous pouvez utiliser dans le cas présent :

```
NAME=exercice1.4

CC=gcc
CFLAGS=-Wall -std=c23

$(NAME): $(NAME).lib.o $(NAME).test.o
 $(CC) $(CFLAGS) $(NAME).lib.o $(NAME).test.o -o $(NAME)
```

```
$(NAME).lib.o: $(NAME).lib.c
 $(CC) $(CFLAGS) -c $(NAME).lib.c
$(NAME).test.o: $(NAME).test.c
 $(CC) $(CFLAGS) -c $(NAME).test.c
```

Vous pouvez lancer la compilation avec make -f le_nom_du_makefile ici make -f exercice1.4.malefile.

Supprimer les -o et relancer la compilation, qu'observez-vous ?

1.5.2.2. Partie 2 : Les conditionnelles

1.5.2.2.1. Exercice 2.1

Écrire un programme qui demande un age et indique si la personne est majeure ou mineure, le programme demande ensuite un montant et applique les règles de gestion suivante, les mineurs ont une remise de 5% sur le prix initial, si le prix initial est supérieur à 300 alors une remise de 5 Euros est appliquée. Quelques exécutions possibles :

```
Quel est le prix de vos achats ?
100.0
Quel est votre age ?
18
Le montant a payer et de : 100.000000
```

```
Quel est le prix de vos achats ?
100
Quel est votre age ?
17
Vous êtes mineur vous disposez d'une remise de 5%
Le montant a payer et de : 95.000000
```

```
Quel est le prix de vos achats ?
1000
Quel est votre age ?
18
Le montant a payer et de : 995.000000
```

```
Quel est votre age ?
17
Vous êtes mineur vous disposez d'une remise de 5%
Le montant a payer et de : 945.000000
```

1.5.2.2.2. Exercice 2.2

Nous allons compter les jours séparant une date de celle de Alan Turing, né le 7 juin 1954. Vous devrez utiliser trois fichiers : exercice2.2.lib.c qui contiendra le code des fonctions, le fichier exercice2.2.lib.h fourni qui contient les signatures des fonctions de exercice2.2.lib.test qui permet de tester.

Pour faciliter la compilation, voici une version augmentée du makefile déjà vu, la commande clean permet de supprimer les fichiers objet et l'exécutable, makefile clean -f exercice2.2.makefile :

```
NAME=exercice2.2

CC=gcc
CFLAGS=-Wall -std=c23

$(NAME): $(NAME).lib.o $(NAME).test.o
 $(CC) $(CFLAGS) $(NAME).lib.o $(NAME).test.o -o $(NAME)
$(NAME).lib.o: $(NAME).lib.c
 $(CC) $(CFLAGS) -c $(NAME).lib.c
$(NAME).test.o: $(NAME).test.c
 $(CC) $(CFLAGS) -c $(NAME).test.c

clean:
 rm -f $(NAME) *.o
```

Nous allons coder les fonctions suivantes dont la signature est définie dans le .h :

```
/exercice2.2.lib.h
/**
```

```
 * Prend en paramètre l'année
 * Renvoie true si l'année est bissextile et false sinon
 */
bool estBissextile(unsigned int);

/**
 * Prend en paramètres l'année et le mois
 * Renvoie le npmbre de jours du mois ou
 * -1 si l'année est antérieur à 1582 ou
 * si le mois n'est pas valide
 * 1 janvier, ..., 12 décembre
 */
short joursDansMois(unsigned int, unsigned short);

/**
 * Prend en paramètre un jour, un mois, une année
 * Renvoie true si la date est valide false sinon
 * les année doivent être supérieure ou égale à 1582
 */
bool estJourValide(unsigned short, unsigned short,unsigned int);

/**
 * Prend en paramètre un jour, un mois, une année
 * Renvoie le nombre de jours depuis le 1 janvier 1582 ou
 * -1 si la date n'est pas valide
 */
int joursDepuisOrigine(unsigned short, unsigned short,unsigned int);
/**
 * Prend en paramètre un jour, un mois, une année
 * Renvoie le nombre de jours depuis le 23 juin 1912
 * -1 si la date n'est pas valide
 */
int jourDepuisNaissanceTuring(unsigned short, unsigned short,unsigned int);
```

Pour rappel, les années sont bissextiles si elles sont multiples de quatre, mais pas si elles sont multiples de cent, à l'exception des années multiples de quatre cents qui, elles, sont également bissextiles. Ainsi, les années 2020, 2024 et 2028 sont bissextiles, de même que 2000 ou 2400, mais pas 1900, 2100, 2200 et 2300.

Nous verrons bientôt les boucles, le code suivant les utilisent et permet de calculer le nombre de jours (nbJours) séparant une date choisie (jour, mois, annee) du premier janvier 1582 :

```
int nbJours = 0;
 for (unsigned int anneeCourante=1582; anneeCourante <annee;anneeCourante++)
  nbJours += estBissextile(anneeCourante) ? 366 : 365;
 for (unsigned int moisCourant=1; moisCourant< mois; moisCourant++)
  nbJours += joursDansMois(annee,moisCourant);
 nbJours += jour  - 1;
```

Le fichier exercice2.2.test.c vous permet de tester vos fonctions.

1.5.2.3. Partie 3 : les boucles

Nous avons qu'il existe trois boucles en C : while, do while et for.

1.5.2.3.1. Exercice 3.1

Écrire un programme en C qui fait la somme des 10 premiers nombres entiers strictement positifs utilisant chacune des 3 boucles.

1.5.2.3.2. Exercice 3.2

Écrire un programme C qui affiche la table de multiplication pour un entier n choisi.

Si n = 3 alors l'affichage sera :

```
3   6   9   12  15  18  21  24  27
```

Puis un programme qui affiche les n tables de multiplication de 1 à n.

Si n = 3 alors l'affichage sera :

```
1   2   3   4   5   6   7   8   9
2   4   6   8   10  12  14  16  18
3   6   9   12  15  18  21  24  27
```

1.5.2.3.3. Exercice 3.3

La banque X nous accorde un prêt si la somme de vos intérêts dépassent 1000 euros. L'intérêt annuel et le remboursement annuel sont choisis. Écrire un programme qui lit un montant, un taux, un remboursement et qui indique le nombre d'années nécessaires pour obtenir le prêt ou affiche un message d'erreur si le prêt ne peut-être obtenu. Quelques exemples de calculs :

```
A0 : Emprunt initial 1000 Euros, taux 0,05, rembousement 100
On pose montantRestant = 1000, annee=0, interets=0

A1 : interets = interets + montantRestant * taux = 0+ 1000*0,05 = 50,
     montantRestant = montantRestant+interets * taux - remboursement=1000+50-100 = 950
     annee = annne + 1 = 1

A2 : interets = 97,5
     montantRestant = 897,5
     annee = 2
...

A15 : interets = 421,07 Euros
      montantRestant <= 0
      annee = 15
Le crédit est refusé
```

```
A0 : Emprunt initial 1000 Euros, taux 0,5, remboursement 500

On ne pourra jamais rembourser le prêt
Le crédit est refusé
```

```
A0 : Emprunt initial 1000 Euros, taux 0,05, remboursement 60
Le prêt est accordé pour 37 ans.
```

1.5.2.4. Partie 4 (optionnelle) : les tableaux

Les exercices qui suivent sont facultatifs, ils permettent à ceux qui avances vite d'aller plus loin.

Un tableau est déclaré somme suit type identificateur[longueur]; Il est également possible de définir une taille implicite définie à l'initialisation.

```
int tab[3] = { 10, 20, 30 };
//ou int tab[] = { 10, 20, 30 };
```

On accède à la valeur d'une case en utilisant sa postion, la première case est numérotée 0.

```
tab[0] = 10;
tab[1] = 20;
tab[3] = 30;
```

Nous le verrons ultérieurement mais tab est un pointeur constant vers le premier élément du tableau.

```
tab[0] = 10;
//ou *tab = 10;
tab[1] = 20;
//ou *(tab+1)=20;
```

1.5.2.4.1. Exercice 4.1

Écrire un programme qui calcul la moyenne d'un tableau d'entiers*L'utilisation de sous programme peut vous aider à tester*..

1.5.2.4.2. Exercice 4.2

Écrire un programme qui inverse les éléments d'un tableau de caractères.

1.5.2.4.3. Exercice 4.3

Les chaîne de caractères sont des tableaux de caractères dont la dernière case contient le caractère \0;

Écrire un programme qui indique la longueur de la chaîne de caractère lue au clavier.

1.5.2.5. Partie 5 (optionnelle) : les pointeurs

Un pointeur est une variable qui permet de stocker une adresse d'un type choisi, les pointeurs disposent d'une arithmétique (addition ou soustraction d'entier, soustraction entre 2 pointeurs, opérateurs de comparaison). Pour rappel & est l'adresse et * la valeur référencée :

```
int i = 8;
int * ptr = &i ;  // ptr pointe i

printf("Valeur de ptr (adresse de i) : %p\n", ptr);
printf("Valeur de ptr+1 (adresse suivant la variable i) : %p\n\n",  ptr+1  );
```

1.5.2.5.1. Exercice 5.1

Que donne le code suivant ?

```
#include<stdio.h>
#include<stdlib.h>
int main()
{
    char a,b,*pa,*pb;
    char tmp;
    pa = &a;
    pb = &b;
    printf("Entrez le premier charactere (a): ");
    scanf("%c",pa);
    getchar(); //utilisé pour consommer le retour à la ligne
    printf("Entrez le deuxieme charactere (b): ");
    scanf("%c",pb);
    printf("a = %c et b = %c.\n",a,b);
    tmp = *pa;
    *pa = *pb;
    *pb = tmp;
    printf("a = %c et b = %c.\n",a,b);
    return 0;
}
```

1.5.2.5.2. Exercice 5.2

Un pointeur permet de modifier la valeur d'une variable passée en paramètre.

Compléter le code de clear(int *p) pour quelle affecte 0 à chaque case du tableau :

```
void clear (int *p) {
  //TODO
}

int main ()
{
   int tab[5];

   clear (tab);

   return 0;
}
```

1.5.2.5.3. Exercice 5.3

En utilisant les pointeurs, écrire un programme qui indique si une chaîne de caractères est un palindrome (Un palindrome est un mot qui reste le même qu'on le lise de gauche à droite ou de droite à gauche).

1.6. Programmation en go

Cette partie est extraite des supports de cours de Loïg Jezequel.

1.6.1. Apprentissage du GO

1.6.1.1. Partie 1 : variables, tableaux, boucles, conditionnelles, fonctions, structures

Ce support a pour but de vous présenter les bases du langage Go. C'est ce langage qui sera utilisé pour réaliser tous les exercices de ce cours d'initiation au développement. Il n'y a pas de prérequis particuliers en dehors des notions de conditionnelle, de boucle et de variable, qui ont été étudiées au lycée.

> **Important**
>
> Dans tout ce support un texte avec cette présentation est un texte contenant une information importante, il faut donc le lire avec attention et veiller à en tenir compte.

Un texte avec cet encadrement est une remarque.

TODO est un travail que vous avez à faire.

> **Important**
>
> Même si la plupart des exercices de ce support peuvent vous sembler très faciles, faites les tous sérieusement : cela vous permettra de retenir la syntaxe du langage Go et d'être ainsi plus à l'aise lors des prochaines séances.

Si vous souhaitez avoir un autre point de vue sur l'apprentissage du langage Go, vous pouvez consulter le tutoriel officiel A Tour of Go [3] ou le site Go by Example [4] en faisant toute fois attention au fait que ces sites sont plutôt faits pour les personnes ayant déjà une bonne expérience de la programmation.

1.6.1.1.1. Un premier programme

Le programme hello.go est un code Go très simple. Avant d'expliquer les différents éléments qui le constituent nous allons voir comment l'exécuter, c'est-à-dire comment le faire fonctionner sur un ordinateur.

Exemple 1.1. hello.go

```
//hello.go
package main

import "fmt"

func main() {
 fmt.Println("Hello World!")
}
```

TODO Exercice 1.1.0 : Récupérez le fichier hello.go et enregistrez-le dans un répertoire de votre choix. Ouvrez un terminal et placez-vous dans ce répertoire. Utilisez enfin la commande go run hello.go. Si tout s'est bien passé vous avez exécuté le programme et le texte Hello World ! s'est affiché dans votre terminal.

Nous allons maintenant analyser ce programme simple.

- La première ligne indique package main. Nous verrons plus tard dans le cours à quoi exactement cela correspond. Pour l'instant il vous suffit de retenir que tous vos fichiers de code Go doivent commencer par cette ligne.

- La deuxième ligne indique import "fmt". Cette ligne permet de récupérer du code déjà écrit par d'autres développeurs pour pouvoir le réutiliser par la suite. Le code récupéré est ici celui de la bibliothèque fmt, qui permet notamment d'afficher des choses dans le terminal.

[3] https://go.dev/tour/welcome/1
[4] https://gobyexample.com/

- La troisième ligne indique func main() { et elle est associée à la dernière ligne } qui ferme l'accolade ouverte auparavant. Nous verrons un peu plus tard en détail à quoi correspond le mot clé func (déclaration d'une fonction) mais pour le moment il vous suffit de retenir que, car cette fonction s'appelle main, le code qui est écrit entre les accolades est celui qui sera exécuté quand on exécutera le programme.
- La quatrième ligne indique fmt.Println("Hello World !"), c'est le code principal du programme. On utilise une fonction Println. Pour savoir que c'est la fonction Println qui provient de la bibliothèque fmt (et pas une autre version de cette fonction) on préfixe le nom de cette fonction de fmt. Cette fonction a ici un argument "Hello World !" et affiche celui-ci.

TODO Exercice 1.1.1 : bonjour Écrivez puis testez un programme qui affiche Bonjour ! dans le terminal où on l'exécute.

On ne va bien sûr pas apprendre par cœur le contenu de toutes les bibliothèques disponibles en Go (il y en a beaucoup). Lorsqu'on a besoin d'une fonctionnalité particulière on cherche dans la documentation du langage [5] *si elle existe. On y trouve notamment la description de toutes les bibliothèques fournies de base avec le langage : la bibliothèque standard* [6] *(qui contient par exemple fmt). Il existe aussi de nombreuses autres bibliothèques, développées par des tiers.*

Important

Dans toute la suite on créera nos programmes sur ce modèle (package main, func main() {}) et on les exécutera en utilisant la commande go run.

1.6.1.1.2. Variables

Quand on écrit un programme informatique on a, en général, besoin d'enregistrer des valeurs pour les réutiliser plus tard. Pour cela on utilise des variables.

1.6.1.1.2.1. Déclaration de variables

Lorsque l'on crée une nouvelle variable on dit qu'on la déclare. Le programme vars.go montre quelques exemples de déclarations de variables en Go.

Exemple 1.2. vars.go

```
package main

import "fmt"

func main() {
 var n int
 fmt.Println(n)

 var b bool
 fmt.Println(b)

 var s string
 fmt.Println(s)
}
```

On peut remarquer que la déclaration d'une variable se fait avec le mot clé var, suivi du nom de la variable, suivi d'un type. Ce type indique la nature des choses qu'on stockera dans la variable. Toute variable a obligatoirement un type qui est fixé une fois pour toute : on ne peut pas le changer après la déclaration.

Dans ce programme on observe trois types différents : int qui représente un entier, bool qui représente un booléen (vrai ou faux) et string qui représente une chaîne de caractères (une séquence de symboles, comme par exemple "Hello World!").

On verra d'autre types par la suite.

[5] https ://golang.org/doc
[6] https ://pkg.go.dev/std

1.6.1.1.2.2. Valeurs par défaut

TODO Exercice 1.2.0 : Récupérez le fichier vars.go puis exécutez-le.

On remarque que, bien qu'on n'ait pas donné de valeurs aux variables, celles-ci en ont tout de même : 0 pour l'entier, false pour le booléen, et "" (c'c'est-à-dire la chaîne vide) pour la chaîne de caractères.

En Go, contrairement à beaucoup d'autres langages, toute variable a une valeur par défaut, qui dépend de son type. Si vous créez un entier, par exemple, vous avez la garantie que, tant que vous ne le modifiez pas, il vaut 0.

1.6.1.1.2.3. Variables inutilisées

TODO Exercice 1.2.1 : Modifiez le fichier vars.go en enlevant une (et une seule) des utilisations de Println puis exécutez-le à nouveau. Vous devez observer une erreur.

L'erreur observée (dans le cas où on supprime l'affichage de b) doit normalement ressembler à ceci : ./vars.go :8 :6 : b declared but not used. Elle indique que, dans le fichier vars.go, à la 6ème lettre de la ligne 8, une variable b est déclarée qui n'est ensuite jamais utilisée dans le reste du programme.

Important

En Go, si on déclare une variable on doit obligatoirement l'utiliser ensuite.
En enlevant toutes les utilisations de Println à vars.go on observe une autre erreur qui nous indique que l'on importe fmt mais que l''on ne l'utilise pas et qu'il faut donc retirer toute la ligne import "fmt".

1.6.1.1.2.4. Affectation de variables

Tout l'intérêt des variables est de pouvoir changer leur valeur, on dit qu'on les affecte. On peut le faire en utilisant le signe = comme le montre le programme vars2.go.

Exemple 1.3. vars2.go

```
package main

import "fmt"

func main() {
 var n int = 1
 fmt.Println(n)
 n = 2
 fmt.Println(n)
}
```

On notera qu'on peut affecter une variable dès sa déclaration var n int = 1 ou plus tard n = 2 et dans ce cas on écrit seulement le nom de la variable à gauche du signe = (on ne souhaite pas la redéclarer).

TODO Exercice 1.2.2 : Récupérez le fichier vars2.go puis exécutez-le.

TODO Exercice 1.2.3 : Modifiez le programme hello.go pour stocker le texte dans une variable avant de l'afficher. Testez votre programme.

1.6.1.1.2.5. Calculs

Enfin, on peut utiliser nos variables pour faire des calculs. On dispose des opérations standards associées au type des variables manipulées, en particulier :

- l'addition (+), la soustraction (-), la multiplication (*), la division (/) et le modulo (%) pour les entiers,
- le OU (||), le ET (&&) et le NON (!) pour les booléens,
- la concaténation (+) pour les chaînes de caractères,

- mais aussi les comparaisons (supérieur strict >, supérieur ou égal >=, inférieur strict <, inférieur ou égal <=, égal ==, différent!=) qui traitent des entiers mais donnent en résultat des booléens.

Il existe d'autres opérations et des raccourcis syntaxiques pour les cas d'utilisations les plus communs de celles-ci, que vous découvrirez au fur et à mesure de votre apprentissage du langage.

Le programme calcul.go donne quelques exemples d'utilisation de ces opérations. On peut noter que ce n'est pas un problème d'écrire n = n+10 : le calcul à droite du signe = est réalisé avant de changer la valeur de la variable à gauche de celui-ci. On utilisera très souvent ce genre de codes où la valeur d'une variable dépend de sa valeur précédente.

Exemple 1.4. calcul.go

```
package main

import "fmt"

func main() {
 var n int = 3 * 5
 n = n + 10
 fmt.Println(n)

 var b bool = true
 b = !b
 fmt.Println(b)

 var s string = "Bon"
 s = s + "jour"
 fmt.Println(s)

 fmt.Println(3 == 5)
 fmt.Println(3 != 5)
 fmt.Println(3 >= 5)
 fmt.Println(3 <= 5)
}
```

TODO : Exercice 1.2.4 : Récupérez le fichier calcul.go puis exécutez-le.

TODO : Exercice 1.2.5 : Écrivez un programme qui, à partir de deux variables contenant les chaînes de caractères "Hello" et "World" utilise la concaténation pour construire la chaîne "Hello World !" puis l'affiche. N'oubliez pas l'espace et le point d'exclamation.

TODO : Exercice 1.2.6 : À l'aide d'un programme simple donnez la valeur de (6x + 4)/2 quand x vaut 5. Modifiez votre programme pour donner cette valeur quand x vaut 7. Seule la valeur initiale d'une variable doit être modifiée pour passer d'un programme à l'autre.

TODO : Exercice 1.2.7 : Écrivez un programme qui affiche true si un nombre entier stocké dans une variable n est strictement plus grand que 5 et qui affiche false sinon.

En Go on a accès à l'opération modulo, qui calcule le reste d'une division euclidienne (une division du entier par un entier). Ainsi n % m calcule le reste de la division de n par m. On a par exemple 7 % 2 qui vaut 1 car 7 = 2 $*$ 3 + 1 (quotient de 3, reste de 1).

TODO : Exercice 1.2.8 : Écrivez un programme qui, étant donné un temps t en seconde, calcule son équivalent en heures/minutes/secondes et l'affiche. Le nombre de secondes et le nombre de minutes dans cet équivalent doivent bien sûr toujours être inférieurs à 60. Testez votre programme avec plusieurs temps t différents afin d'avoir confiance dans le fait qu'il est correct.

Il existe d'autres façon de déclarer une variable en Go. À la place de faire var n int = 1 on peut faire var n = 1 ou encore n := 1. Ceci utilise un mécanisme d'inférence de type : Go devine quel doit être le type de la variable n à partir de la valeur que l'on donne à celle-ci. Comme il est important d'être bien conscient des types des variables que l'on manipule, il vous est demandé de ne pas utiliser ces manières de déclarer les variables dans les cours d'initiation au développement.

En plus de déclarer des variables on peut, en Go, déclarer des constantes. Pour cela on remplace simplement var par const. Les constantes fonctionnent comme les variables, mais on est obligé de les affecter à la déclaration (pas de valeur par défaut) et on ne peut plus les modifier ensuite.

1.6.1.1.3. Tableaux

Un tableau est une variable particulière qui contient n valeurs indexées par des entiers entre 0 et n_ 1. Ces valeurs doivent toutes être du même type. Cela peut être utile, par exemple, pour représenter des objets mathématiques, comme les vecteurs ou les matrices, ou pour représenter un ensemble ordonné de valeurs.

1.6.1.1.3.1. Déclaration

Pour déclarer un tableau on peut simplement créer une variable dont le type indique le type des valeurs qui seront contenues dans ce tableau, précédé de crochets. Par exemple, pour un tableau d'entiers, [ ]int. Le programme tabs.go donne quelques exemples de déclarations de tableaux.

Exemple 1.5. tabs.go

```
package main

import "fmt"

func main() {
 var tint []int
 var tbool []bool
 var tstring []string
 fmt.Println(tint, tbool, tstring)
}
```

TODO Exercice 1.3.0 : Récupérez le fichier tabs.go puis exécutez-le. Quelle est la valeur par défaut d'une variable de type tableau?

1.6.1.1.3.2. Accès aux valeurs et affectations

Comme pour les autres variables on peut bien sûr affecter une valeur à une variable de type tableau en utilisant le signe =. Il faut cependant utiliser une syntaxe un peu plus complexe que pour les types de base pour représenter la valeur du tableau, comme on peut le voir dans le programme tabs2.go.

Exemple 1.6. tabs2.go

```
package main

import "fmt"

func main() {
 var tint []int = []int{1, 2, 3, 1000, 27}
 var tbool []bool = []bool{true, false, false}
 fmt.Println(tint, tbool)
}
```

TODO Exercice 1.3.1 : Récupérez le fichier tabs2.go et exécutez-le. Modifiez-le ensuite pour affecter et afficher un tableau de chaînes de caractères contenant, dans cette ordre, les chaînes "Bonjour", "le" et "monde". Testez votre modification.

On peut aussi accéder à une valeur dans une case d'un tableau. Pour cela on va indiquer entre crochets après le nom du tableau le numéro de cette case. Le programme tabs3.go montre ceci. On voit aussi que l'on peut affecter une valeur à une case d'un tableau sans modifier les autres en accédant à cette valeur et en utilisant le signe =.

Exemple 1.7. tabs3.go

```
package main

import "fmt"

func main() {
```

```
 var tint []int = []int{1, 2, 3, 1000, 27}
 fmt.Println(tint[0])
 fmt.Println(tint[3])
 fmt.Println(tint)
 tint[3] = 0
 fmt.Println(tint)
}
```

Important

Attention, les cases d'un tableau sont numérotées à partir de 0 : la première case est la case 0 et pas la case 1.

TODO Exercice 1.3.2 Récupérez le fichier tabs3.go et exécutez-le. Modifiez ce programme pour afficher la valeur de la dernière case du tableau. Testez votre code. Modifiez à nouveau le programme pour essayer d'afficher une case du tableau qui n'existe pas (la case numéro 50 par exemple). Testez votre code et analysez l'erreur qui s'affiche pour bien la comprendre.

1.6.1.1.3.3. Taille

Pour éviter de dépasser les limites d'un tableau, il peut être utile de connaître sa longueur, c'est- à-dire son nombre de cases. On peut faire ceci en Go à l'aide de la fonction len, comme le montre le programme tab4.go.

Exemple 1.8. tabs4.go

```
package main

import "fmt"

func main() {
 var tint []int = []int{1, 2, 3, 1000, 27}
 fmt.Println(len(tint))
 tint = []int{4, 5, 6}
 fmt.Println(len(tint))
}
```

TODO Exercice 1.3.3 : Récupérez le fichier tabs4.go et exécutez-le. Comment peut-on avoir le numéro de la dernière case d'un tableau t à partir du résultat de len(t)? Qu'est-ce-que cela veut dire si on trouve que le numéro de cette case est -1 ?

1.6.1.1.4. Boucles

Lorsque l'on exécute un programme Go tels qu'on les a vus jusqu'à'à présent, les lignes de code de la fonction main sont exécutées une à une, dans l'ordre où elles sont écrites, puis le programme s'arrête. Ceci ne permet pas de réaliser des calculs très intéressants (comment faire, par exemple, pour afficher l'un après l'autre tous les éléments contenus dans un tableau). On utilisera donc, en plus, des structures de contrôle qui vont permettre d'exécuter certaines parties de notre code plus ou moins de fois et dans des ordres différents en fonction de conditions sur les valeurs de variables ou sur les résultats de calculs. Ces structures de contrôle sont de deux types : les conditionnelles (que nous verrons un peu plus tard) et les boucles.

1.6.1.1.4.1. La boucle for, utilisation de base

Selon les langages de programmation qu'on utilise on a accès à plus ou moins de différents types de boucles (for, while, do while, etc), cependant, en Go, il n'existe qu'un seul type de boucle : la boucle for.

Le programme for.go montre un premier exemple de boucle for, dans sa forme la plus simple. On peut schématiser cette boucle ainsi : for initialisation ; condition ; mise à ajour {code} où les différents éléments ont les caractéristiques suivantes :

Exemple 1.9. for.go

```
package main

import "fmt"
```

```
func main() {
 for i := 0; i < 10; i = i + 2 {
  fmt.Println(i)
 }
}
```

initialisation

Création d'une variable de boucle, ici la variable est i et est initialisée à 0 par la code i := 0.

condition

Condition sur la valeur de la variable de boucle qui indique quand la boucle doit continuer à s'exécuter. Ici, la boucle s'exécute tant que i est strictement plus petit que 10, ce qui est exprimé par le code i < 10.

mise à jour

Changement appliqué à la variable de boucle à chaque itération de celle-ci. Ici on ajoute 2 à i, ce qui est représenté par le code i = i + 2.

code

Code qui est exécuté à chaque itération de la boucle.

En d'autres terme, cette boucle va initialiser une variable i à 0, puis exécuter le code fmt.Println(i), puis changer la valeur de i en lui ajoutant 2, exécuter à nouveau le code, ajouter 2 à i, et ainsi de suite jusqu'à ce que en ajoutant 2 à i sa valeur devienne supérieure ou égale à 10.

TODO Exercice 1.4.0 : Sans l'exécuter, indiquez ce qu'affiche le programme for.go. Récupérez ensuite le fichier for.go et exécutez-le pour vérifier votre réponse.

TODO Exercice 1.4.1 : Écrivez un programme qui affiche tous les multiples de 5 compris entre 10 et 100 inclus. Testez votre programme pour vous assurer qu'il est correct.

TODO Exercice 1.4.2 : Écrivez un programme qui, à l'aide d'une boucle, affiche 10 fois Bonjour.

TODO Exercice 1.4.3 : Écrivez un programme qui déclare un tableau d'entiers contenant 10 valeurs de votre choix puis qui, à l'aide d'une boucle for, affiche tous ces entiers dans l'ordre du tableau. Votre variable de boucle servira à déterminer le numéro de la case dont la valeur doit être affichée à chaque tour de boucle. La fonction len vous permettra d'arrêter votre boucle au bon moment. Testez votre programme. Modifiez la déclaration de votre tableau pour qu'il contienne maintenant 12 valeurs. Sans rien modifier d'autre, votre programme doit toujours fonctionner. Testez-le. Changez maintenant votre tableau pour qu'il ne contienne aucune valeur, votre programme doit toujours fonctionner (il n'affiche pas de valeurs puisque le tableau est vide, mais il ne doit pas non plus afficher d'erreur). Testez-le.

En écrivant le programme de l'exercice précédent, vous avez sans doute mis-à-jour la variable de boucle en écrivant i = i + 1. Il existe un raccourci d'écriture pour cette opération : i++

TODO Exercice 1.4.4 : Modifiez le programme for.go pour afficher la valeur de i après la fin de la boucle. Testez-le et expliquez l'erreur que vous observez.

1.6.1.1.4.2. Variable de boucle déclarée à l'extérieur

On a vu, avec le dernier exercice, qu'une fois la boucle terminée, on ne pouvait plus accéder à la valeur de la variable de boucle. Ceci est dû au fait qu'une variable a une durée de vie limitée (pour économiser des ressources mémoire). Dans le cas de la boucle for précédente, la variable a été déclarée dans l'initialisation de la boucle, sa durée de vie est celle de la boucle, et par conséquent elle n'existe plus une fois que celle-ci est terminée. Si on a besoin d'accéder à une variable de boucle après la fin de celle-ci, il est possible en Go de déclarer cette variable en dehors de la boucle, comme le montre le programme for2.go.

Exemple 1.10. for2.go

```
package main
```

```
import "fmt"

func main() {
 var i int
 for i = 0; i < 10; i = i + 2 {
  fmt.Println(i)
 }
 fmt.Println(i)
}
```

TODO Exercice 1.4.5 : Récupérez le programme for2.go et testez-le.

Si on déclare une variable dans le code exécuté par une boucle, cette variable est redéclaré à chaque tour de boucle : sa durée de vie est celle d'une itération. Pour qu'une variable puisse être mise à jour par une boucle il faut donc aussi la déclarer à l'extérieur de celle-ci.

TODO : Exercice 1.4.6 Écrivez un programme qui déclare un tableau d'entiers contenant 10 valeurs de votre choix puis qui, à l'aide d'une boucle for, calcule la somme de ces 10 entiers. Testez-le. Modifiez la déclaration de votre tableau pour qu'il contienne maintenant 12 valeurs. Sans rien modifier d'autre, votre programme doit maintenant calculer la somme des 12 entiers. Testez-le.

1.6.1.1.4.3. Intermède : le type float64

En Go il existe bien sûr un type pour représenter les nombre décimaux. Il s'agit du type float64 (il existe en fait aussi float32). On peut donc déclarer une variable x contenant une valeur décimale en écrivant var x float64. Et on peut lui attribuer une valeur comme pour les autres types (la virgule se représente par un point) : x = 12.5 par exemple.

TODO Exercice 1.4.7 : Écrivez un programme vous permettant de trouver quelle est la valeur par défaut d'une variable de type float64.

TODO Exercice 1.4.8 Écrivez un programme qui à partir d'un tableau contenant les notes sur 20 d'un étudiant au cours d'un semestre calcule la moyenne de cet étudiant.

1.6.1.1.4.4. Boucle avec seulement une condition

La condition utilisée pour définir la fin d'une boucle n'est en fait pas obligée de concerner la variable de boucle. On n'est d'ailleurs pas obligé d'avoir une variable de boucle. Le programme for3.go montre un exemple de boucle for sans variable de boucle, cette boucle n'a donc qu'une condition (pas d'initialisation et pas de mise à jour).

Exemple 1.11. for3.go

```
package main

import "fmt"

func main() {
 var reste int = 32
 var quotient int = 0
 const diviseur int = 4
 for reste >= diviseur {
  reste = reste - diviseur
  quotient++
 }
 fmt.Println(quotient)
}
```

TODO Exercice 1.4.9 : Récupérez le fichier for3.go. Exécutez-le plusieurs fois en modifiant les valeurs affectées initialement aux variables reste et diviseur. Que calcule ce programme ?

TODO Exercice 1.4.10 : Écrivez sur le même modèle un programme qui calcule le produit de deux variables entières sans utiliser l'opération de multiplication.
On peut convertir un entier x en flottant en écrivant float64(x).

TODO Exercice 1.4.11 : Lors de la naissance de sa petite fille Anaïs, son grand-père lui a ouvert un compte épargne en y déposant 100C. À chaque anniversaire d'Anaïs, il verse sur ce compte un montant correspondant à 10 fois l'age qu'elle vient d'avoir. (Par exemple, à 5 ans, Anaïs va recevoir 50C.) En outre, le compte épargne d'Anaïs a un taux fixe de 3,5% dont les intérêts sont versés à chaque date anniversaire. Écrivez un programme qui étant donné un montant en euros, indique à Anaïs l'age qu'elle devra avoir pour en disposer sur son compte.

La syntaxe des boucles en Go est en fait un peu plus générale que ce que nous avons vu ici. On peut par exemple écrire une boucle infinie en laissant vide l'espace réservé à la condition de boucle : for {code}.

1.6.1.1.5. Conditionnelles

Nous allons maintenant nous intéresser à un deuxième type de structures de contrôle : les conditionnelles. Ces structures permettent de faire des choix en fonction de valeurs de variables ou de résultats de calculs.

1.6.1.1.5.1. if

Exemple 1.12. if.go

La conditionnelle la plus simple est le if, elle permet d'exécuter un code parmi deux en fonction d'une condition. Le programme if.go montre un exemple d'utilisation de cette conditionnelle.

```
package main
import" fmt "
func main ( ) {
var n i n t = 10
i f n < 5 {
fmt . Println ( " Moins } else {
fmt . Println ( " 5 ou }
que 5 " )
plus " )
}
```

Le schéma général du if est le suivant : if condition {code1} else {code2}. Si la condition est vérifiée (dans l'exemple, si n est strictement inférieur à 5) alors on exécute le code code1 (ici, afficher moins que 5). Si la condition n'est pas vérifiée (dans l'exemple, si n est supérieur ou égal à 5) alors on exécute le code code2 (ici, afficher 5 ou plus). On appellera code1 et code2 les branches de la conditionnelle (qui est parfois appelée branchement).

TODO Exercice 1.5.0 : Récupérez le fichier if.go et testez-le. Modifiez la valeur de n pour constater que le programme peut bien exécuter le code de l'une ou l'autre des branches de la conditionnelle.

TODO Exercice 1.5.1 : Un magasin propose à ses clients une remise de 5% (arrondi à l'entier inférieur) sur le mon- tant total des achats lorsque celui-ci dépasse 300C. Écrivez un programme qui à partir d'un montant d'achat en euros (stocké dans une variable entière) affiche la somme à payer par le client. Testez votre programme pour différents montants d'achats (en particulier des montants inférieurs à 300C et des montants supérieurs à 300C)

TODO Exercice 1.5.2 : Écrivez un programme qui déclare trois variables entières puis trouve et affiche la valeur de la plus grande d'entre-elles. Testez-le pour différentes valeurs des variables.

TODO Exercice 1.5.3 : Écrivez un programme qui déclare deux variables entières puis indique si leur produit est positif, négatif ou nul sans calculer ce produit. Testez-le pour différentes valeurs des variables.

La partie else est facultative, si on l'omet le code est exécutée si la condition est vérifiée et il ne se passe rien sinon. Le programme if2.go en donne un exemple.

Exemple 1.13. if2.go

```
package main

import "fmt"

func main() {
 var n int = 10
 if n < 5 {
  fmt.Println("Moins que 5")
```

```
 }
 fmt.Println("Au revoir")
}
```

Dans tous les cas le programme affiche Au revoir, par contre il n'affiche Moins que 5 que si n est strictement inférieur à 5.

TODO Exercice 1.5.5 : Récupérez le fichier if2.go et testez-le. Modifiez la valeur de n pour constater ce qu'il se passe lorsque la condition est vraie et lorsqu'elle est fausse.

TODO Exercice 1.5.6 : Écrivez un programme qui déclare un tableau d'entiers puis qui, à l'aide d'une boucle for, affiche ceux qui sont strictement inférieurs à 5, dans l'ordre du tableau. Testez votre programme pour plusieurs tableaux de tailles et de contenus différents.

TODO Exercice 1.5.7 : Écrivez un programme qui déclare un entier n et un tableau d'entiers puis qui compte (et affiche) combien de fois l'entier n apparaît dans le tableau. Testez votre programme pour plusieurs tableaux de tailles et de contenus différents, ainsi que pour plusieurs valeurs de n.

1.6.1.1.5.2. Retour sur les boucles

On peut stopper une boucle de manière anticipée en utilisant le mot clé break : au moment où ce mot clé est rencontré dans le code de la boucle, celle-ci se termine immédiatement. Ceci est particulière- ment utile en combinaison avec des conditionnelles. Le programme break.go montre un exemple d'utilisation de break.

```
package main

import "fmt"

func main() {
 var n int = 10
 var p int = 5
 for i := 0; i < n; i++ {
  fmt.Println("L'itération", i, "commence")
  if i == p {
   break
  }
  fmt.Println("L'itération", i, "termine")
 }
}
```

Dans ce programme, n tours de boucle devraient avoir lieu. Cependant, au sixième tour, quand la variable i vaut p, la condition de la conditionnelle est vérifiée. On exécute donc le code de la première (et seule) branche de cette conditionnelle, qui est un break, ce qui fait terminer la boucle.

TODO Exercice 1.5.8 : Récupérez le fichier break.go et exécutez-le. Faites quelques tests en modifiant n et p, pour bien comprendre le fonctionnement de break.

TODO Exercice 1.5.9 : Écrivez un programme qui déclare un tableau d'entiers, puis affiche une à une, dans l'ordre, les valeurs contenues dans ce tableau à l'aide d'une boucle en s'arrêtant dès qu'il rencontre la valeur 3. Testez-le avec des tableaux contenant la valeur 3 et des tableaux ne la contenant pas.

Dans le même ordre d'idée, on peut stopper l'itération en cours d'une boucle (et passer directement au début de la suivante) en utilisant le mot clé continue.

TODO Exercice 1.5.10 Remplacez le mot clé break par le mot clé continue dans le programme du fichier break.go. Testez-le pour différentes valeurs de n et p.

1.6.1.1.5.3. switch

Si on veut faire un choix entre plus de deux alternatives, on peut imbriquer des conditionnelles en enchaînant directement un if après un else comme le montre le programme if3.go.

Exemple 1.14. if3.go

```
package main
```

```
import "fmt"

func main() {
 var n int = 10
 if n < 5 {
  fmt.Println("Moins que 5")
 } else if n <= 10 {
  fmt.Println("Entre 5 et 10")
 } else {
  fmt.Println("Plus de 10")
 }
}
```

TODO Exercice 1.5.11 : Récupérez le fichier if3.go et testez-le pour différentes valeurs de n, de façon à observer les trois affichages possibles.

Si on a beaucoup d'alternatives, on se rend bien compte que cette imbrication de conditionnelles peut devenir peu pratique à écrire. On peut alors la remplacer par un switch, qui est une autre structure de contrôle. Le programme 16 est une réécriture du programme switch.go à l'aide d'un switch.

Exemple 1.15. switch.go

```
package main

import "fmt"

func main() {
 var n int = 10
 switch {
 case n < 5:
  fmt.Println("Moins que 5")
 case n <= 10:
  fmt.Println("Entre 5 et 10")
 default:
  fmt.Println("Plus de 10")
 }
}
```

TODO Exercice 1.5.12 : Récupérez le fichier switch.go et testez-le pour les mêmes valeurs de n que le programme précédent. Constatez que ces deux programmes semblent bien donner les mêmes résultats.*Un switch teste les conditions de ses différents case une à une dans l'ordre et exécute uni- quement le code correspondant à la première de ces conditions qui est vérifiée.*

1.6.1.1.6. Fonctions

On a utilisé dans le début de ce document le nom de fonction sans vraiment dire de quoi il s'agissait. Une fonction est un bout de code auquel on donne un nom et qui, à partir d'arguments va retourner des résultats ou valeurs de retour. Un intérêt des fonctions est d'éviter de réécrire plusieurs fois un même code (ce qui peut être source d'erreurs). La fonction main est une fonction particulière : elle n'a pas d'arguments et ne retourne pas de résultats. De plus, elle est automatiquement appelée au début de l'exécution du programme.

1.6.1.1.6.1. Déclaration de fonction

Pour déclarer une fonction à un argument et une valeur de retour on utilise le mot clé func suivi du nom de la fonction, du nom et type de son argument entre parenthèses, du type de sa valeur de retour, puis d'accolades qui entourent le code de cette fonction. Pour utiliser cette fonction (on dit qu'on l'appelle) on écrit simplement son nom suivi de la valeur qu'on souhaite utiliser pour son argument entre parenthèses. Le programme func.go déclare une telle fonction f puis l'utilise à l'intérieur de la fonction main avec différentes valeurs de son argument.

Exemple 1.16. func.go

```
package main

import "fmt"
```

```
func f(x int) bool {
 return x < 10
}

func main() {
 fmt.Println(f(3))
 fmt.Println(f(5))
 fmt.Println(f(7))
 fmt.Println(f(11))
 fmt.Println(f(13))
}
```

L'argument de la fonction f est un entier qui s'appelle x : à l'intérieur du code de la fonction, quand on fait référence à x cela correspond à la valeur de cet argument. Cette fonction retourne (avec le mot clé return) un booléen. Quand on utilise, par exemple, f(3) cela correspond à exécuter le code de la fonction f avec x qui vaut 3, jusqu'à obtenir la valeur du return et à remplacer f(3) par cette valeur.

TODO Exercice 1.6.0 : Récupérez le fichier func.go puis exécutez-le. Modifiez la valeur du return pour voir l'effet que cela a sur le programme. Essayez aussi d'appeler la fonction avec d'autres valeurs de son argument.

TODO Exercice 1.6.1 Dans un nouveau programme, déclarez une fonction qui prend en paramètre un tableau d'en- tiers et retourne la valeur de la case 3 de ce tableau. Dans le main utilisez cette fonction avec plusieurs tableaux différents, en affichant à chaque fois le résultat obtenu.

1.6.1.1.6.2. Fonctions à plusieurs arguments

Une fonction peut avoir plusieurs arguments, on les sépare par des virgules lors de la déclaration de la fonction et on sépare leurs valeurs par des virgules (en respectant leur ordre de déclaration) lors de l'appel de la fonction. Le programme func2.go donne un exemple de déclaration et d'utilisation d'une fonction g à deux arguments.

Exemple 1.17. func2.go

```
package main

import "fmt"

func g(x int, y int) bool {
 return x < y
}

func main() {
 fmt.Println(g(3, 4))
 fmt.Println(g(4, 3))
 fmt.Println(g(5, 5))
}
```

Les arguments de g sont deux entiers qui s'appellent x et y. Cette fonction retourne un booléen qui indique si x est strictement plus petit que y ou pas.

TODO Exercice 1.6.1 : Récupérez le fichier func2.go puis exécutez-le. Essayez d'appeler la fonction avec d'autres valeurs de ses arguments.

TODO Exercice 1.6.2 : Dans un nouveau programme, déclarez une fonction qui prend en paramètre un tableau d'entiers et un entier n et retourne la valeur de la case n de ce tableau. Dans le main utilisez cette fonction avec plusieurs tableaux différents et plusieurs valeurs de n différentes, en affichant à chaque fois le résultat obtenu.

TODO Exercice 1.6.3 : Deux nombres entiers positifs n et m sont dits amicaux si la somme des diviseurs de n (excepté n) vaut m et si la somme des diviseurs de m (excepté m) vaut n. Écrivez une fonction qui calcul et retourne la somme des diviseurs d'un nombre entier positif donné en argument. Testez cette fonction sur plusieurs nombres en n'oubliant pas de tester les cas particuliers : 0, nombres premiers, etc. Écrivez ensuite une fonction qui, à partir de la fonction précédente, indique (par un booléen) si deux nombres entiers positifs qui lui sont donnés en arguments sont amicaux ou pas. Testez cette fonction sur plusieurs couples de nombres.*On peut bien sûr avoir plus de deux arguments pour une fonction.*

1.6.1.1.6.3. Fonctions retournant plusieurs valeurs

Une fonction peut aussi retourner plusieurs valeurs, comme le montre le programme func3.go.

Exemple 1.18. func3.go

```
package main

import "fmt"

func g(x int, y int) (bool, int) {
 return x < y, x + y
}

func main() {
 var b bool
 var n int
 b, n = g(3, 4)
 fmt.Println(b)
 fmt.Println(n)
}
```

Les types des valeurs de retour (ici bool et int) sont indiqués entre parenthèses et séparés par des virgules à la place habituelle du type de la valeur de retour. Le retour lui même est fait en indiquant les valeurs, dans le même ordre que leurs types ont été indiqués, séparées par des virgules (ici x < y et x + y). On peut récupérer ces valeurs dans des variables (ici b, et n) en les mettant à gauche d'un =, séparées par des virgules.

TODO Exercice 1.6.5 : Récupérez le fichier func3.go sur puis exécutez-le. Essayez d'appeler la fonction avec d'autres valeurs de ses arguments.

1.6.1.1.7. Types structurés

Les types de base (int, float, string, bool, etc) ne sont pas toujours suffisants pour représenter toutes les données qu'on voudrait manipuler. On a vu que l'on peut réunir des éléments d'un même type au sein de tableaux (et on verra plus tard aussi d'autres moyens de le faire), mais il pourrait être intéressant de réunir des éléments de types différents, ou d'avoir des structures plus complexes. Le mot clé struct permet de définir des nouveaux types, constitués de différents champs qui peuvent chacun être d'un type différent. Le programme struct.go donne un exemple de création d'un nouveau type cuple qui contient deux champs de type int, nommés first et second.

Exemple 1.19. struct.go

```
//struct.go
package main

import "fmt"

type cuple struct {
 first  int
 second int
}

func main() {
 var c cuple
 fmt.Println(c)
}
```

TODO Exercice 1.7.0 : Récupérez le fichier struct.go et exécutez-le. Quelle est la valeur par défaut d'une variable de type cuple? Quelle doit être la valeur par défaut d'une variable d'un type structuré quelconque ?

Il est bien sûr possible de fixer soit même les valeurs des champs d'une variable d'un type structuré lorsqu'on la crée. Le programme struct2.go montre comment faire cela.

Exemple 1.20. struct2.go

```
//struct2.go
```

```
package main

import "fmt"

type cuple struct {
 first  int
 second int
}

func main() {
 var c1 cuple = cuple{first: 1, second: 2}
 var c2 cuple = cuple{first: 1}
 var c3 cuple = cuple{second: 2}
 fmt.Println(c1, c2, c3)
}
```

TODO Exercice 1.7.1 : Récupérez le fichier struct2.go et exécutez-le. Essayez de donner d'autres valeurs aux champs. Que se passe-t-il lorsqu'on ne définit pas les valeurs de tous les champs ? (quelle est la valeur des autres champs?)

On n'est pas obligé de donner les noms des champs lorsqu'on fixe la valeur d'une variable d'un type structuré (dans le programme ci-dessus on pourrait écrire var c1 cuple = cuple{1, 2}), mais on doit alors obligatoirement définir tous les champs et les définir dans l'ordre où ils sont déclarés dans la structure.

Enfin, on peut accéder indépendamment aux différents champs d'une variable d'un type structuré, et les modifier. Le programme struct3.go montre comment faire cela.

Exemple 1.21. struct3.go

```
//struct3.go
package main

import "fmt"

type cuple struct {
 first  int
 second int
}

func main() {
 var c cuple = cuple{first: 1, second: 2}
 fmt.Println(c, c.first, c.second)

 c.first = 3
 fmt.Println(c, c.first, c.second)
}
```

TODO Exercice 1.7.2 : Récupérez le fichier struct3.go et exécutez-le. Modifiez le programme pour changer la valeur du champs second et testez-le à nouveau.

Quand on définit un type structuré dans un paquet, si le nom du type commence par une majuscule, alors ce type sera utilisable dans d'autres paquets (à condition d'importer tout ce qu'il faut). De même, les noms des champs ne seront visibles dans d'autres paquets que s'ils commencent par des majuscules. Ainsi, on peut avoir un type dont certains champs sont accessibles à l'extérieur du paquet où il est définit, mais d'autres champs ne sont accessibles que dans ce paquet. Ceci permet de n'exporter que les éléments qui sont pertinents. Avec type on peut aussi donner des noms à des types déjà existants, par exemple type myint int définit un type myint qui correspond au type int. Ceci sera utile en lien avec la partie suivante sur les méthodes.

1.6.1.2. Partie 2 : pointeurs, tableaux, arguments

1.6.1.2.1. Les pointeurs

Les pointeurs sont des variables avec un type particulier et qui permettent de stocker l'endroit ou des valeurs sont situées en mémoire (leur adresse mémoire) plutôt que directement ces valeurs. Nous allons voir comment fonctionnent les pointeurs en Go et à quoi ils peuvent servir.

1.6.1.2.1.1. Passage d’arguments par référence et passage d’arguments par valeur

Si on considère une fonction déclarée par func f(x int, y int), on appelle x et y les arguments de cette fonction. Dans un langage de programmation qui offre la possibilité d’avoir des fonctions (comme le Go) on peut imaginer essentiellement deux mécanismes pour donner des arguments lors d’un appel de fonction (on parle de passer les arguments à la fonction). Pour simplifier la présentation de ces mécanismes, on peut considérer que chaque variable d’un programme est une boîte qui contient une valeur.

1.6.1.2.1.1.1. Passage d’arguments par valeur

Si les arguments sont passés par valeur cela signifie que lorsqu’on appelle notre fonction f en écrivant f(n, m) ce sont les valeurs contenues dans les boîtes n et m qui vont être utilisées par la fonction et pas les boîtes elles-mêmes. Ceci est sans doute plus clair sur un exemple. Si le passage des arguments se fait par valeur, alors à la fin du programme valeur-reference.go la valeur de n n’aura pas changé : appeler f(n, m) ou f(1, 2) ne fait aucune différence.

Exemple 1.22. valeur-reference.go

```
package main

func f(x int, y int) {
 x = x + y
}

func main() {
 var n int = 1
 var m int = 2
 f(n, m)
}
```

1.6.1.2.1.1.2. Passage d’arguments par référence

Si les arguments sont passés par référence, lors d’un appel à f(n, m) on passe à la fonction directement les boîtes n et m. Et donc cette fonction peut modifier le contenu des boîtes, ce qui lui permet de changer la valeur d’une variable qu’on lui passe en argument. Dans ce cas, à la fin du programme valeur-reference.go la variable n aura changé, elle contiendra maintenant la valeur 3.

TODO Exercice 2.1.0 : Proposez un moyen de déterminer si, en Go, le passage d’arguments se fait par valeur ou par référence (vous pouvez pour cela récupérer le fichier valeur-reference.go et mettez-le en oeuvre.

1.6.1.2.1.2. Pointeurs

Le passage d’arguments par référence est quand même parfois utile. Pour y avoir accès on utilise ce que l'on appelle des pointeurs.

Pour représenter les pointeurs on utilise le symbole *, que l'on peut associer à n’importe quel type. Ainsi, par exemple, *int est le type d’un pointeur vers un int et *bool celui d’un pointeur vers un bool.Si on reprend l’analogie des boîtes, une variable de type int est une boîte qui contient un entier alors qu'une variable de type *int est une boîte qui contient une boîte qui elle-même contient un entier.

Si n est une variable vous savez accéder à son contenu, il suffit d’écrire son nom. Mais, si on souhaite accéder à l’entier correspondant à a de type *int c’est un peu plus compliqué : il faut récupérer la valeur qui est dans la boîte contenue dans a. Pour cela on écrit *a et on dit que l'on déréférence le pointeur a. Parfois, on peut aussi avoir besoin d’accéder à la boîte correspondant à une variable n, par exemple pour la mettre dans une autre boîte. Pour cela on peut écrire &n.

Le programme ??? donne quelques exemples d’utilisation.

Exemple 1.23. pointeurs.go

```
package main
```

```
import "fmt"

func main() {
 var n int = 5
 var a *int = &n
 fmt.Println(n, *a)

 n = 6
 fmt.Println(n, *a)

 var m int = 7
 a = &m
 fmt.Println(n, *a, m)
}
```

L'analogie des boîtes a ses limites et n'est pas toujours totalement intuitive : il est tout à fait possible d'avoir deux pointeurs qui indiquent la même adresse mémoire, c'est-à-dire deux boîtes qui contiennent la même boîte.

Le ruban mémoire est une autre analogie : adresse, valeur, nom(s).

```
Adresse  valeur nom
0X000A   5      n, *a
0X000E   0X000A a
```

TODO Exercice 2.1.1 : Récupérez le fichier pointeurs.go et exécutez-le. En utilisant l'analogie d'un ruban mémoire, représentez ce qu'il se passe lors de l'exécution de ce programme.

TODO Exercice 2.1.2 : Quelle est la valeur par défaut d'une variable de type *int ? Que se passe-t-il quand on essaye de déréférencer un tel pointeur ? Expliquez cela à l'aide de l'analogie des boîtes.

Si on a une variable a de type *int on peut bien sûr aussi modifier le valeur contenue dans la boîte contenue dans a. Il suffit pour cela d'utiliser la notation *a, permettant d'accéder à cette valeur, à gauche d'un signe =, par exemple *a = 5.

TODO Exercice 2.1.3 : Modifiez le fichier pointeurs.go pour, après le programme actuel, mettre la valeur 12 dans la boîte contenue dans a puis afficher la valeur contenue dans cette boîte. Testez-votre pro- gramme. Affichez après cela la valeur contenue dans m. Que remarquez-vous ? Expliquez cette observation à l'aide de l'analogie des boîtes.

Puisque on peut avoir un pointeur vers un objet de n'importe quel type, on peut aussi avoir un pointeur vers un pointeur (et un pointeur vers un pointeur vers un pointeur, etc). Il suffit pour cela de rajouter une étoile devant : **int est une boîte qui contient une boîte, qui contient une boîte, qui contient un entier. Le programme ??? donne un exemple de cela.

Exemple 1.24. pointeur2.go

```
package main

import "fmt"

func main() {
 var n int = 5
 var a *int = &n
 var b **int = &a
 var c ***int = &b
 fmt.Println(n, *a, **b, ***c)

 ***c = 7
 fmt.Println(n, *a, **b, ***c)
}
```

TODO Exercice 2.1.4 : Récupérez le ficher pointeurs2.go puis exécutez-le. Expliquez ce que vous observez à l'aide de l'analogie du ruban mémoire.

1.6.1.2.1.3. Pointeurs et passage d'arguments par référence

Une utilité importante des pointeurs est de permettre de simuler le passage d'arguments par ré- férence. Dans le programme valeur-reference2.go l'effet de la fonction fval sur n est exactement le même que celui de la fonction fref sur m. Cependant, cette deuxième tire profit des pointeurs pour simuler un passage d'arguments par référence.

Exemple 1.25. valeur-reference2.go

```
package main

func fval(x int) int {
 return 2*x + 1
}

func fref(a *int) {
 *a = 2*(*a) + 1
}

func main() {
 var n int = 5
 n = fval(n)

 var m int = 5
 fref(&m)
}
```

TODO Exercice 2.1.5 : Récupérez le fichier valeur-reference2.go et exécutez-le. Ajoutez éventuellement des affichage (fmt.Println) pour mieux voir ce qui se passe. Expliquez ce qui se passe lors des deux appels de fonctions en utilisant l'analogie des boîtes.

TODO Exercice 2.1.6 : Récupérez le fichier unefonction.go , lisez son code puis exécutez-le. Modifiez la fonction f et ses appels de façon à simuler le passage d'arguments par référence (sur le modèle de l'exemple précédent avec fval et fref). Testez votre code pour vérifier que les affichages restent les mêmes.

Déréférencer un pointeur à un coût supérieur au simple accès à la valeur d'une variable. Cependant, passer les arguments par valeurs a aussi un coût (celui de la copie des valeurs) qui peut être important pour certains types un peu complexes (nous verrons leur utilité et comment en définir soit-même plus tard).

1.6.1.2.2. Retour sur les structures : méthodes

On appelle méthode une fonction particulière qui est fortement associée à un type. En pratique, on pourrait remplacer toute méthode par une fonction classique mais les méthodes permettent de mieux structurer le code, en associant des fonctionnalités à des types.

Pour définir une méthode on fait comme pour une fonction normale, mais le type sur lequel elle doit s'appliquer est indiqué avant le nom de la fonction. Le programme methode.go donne un exemple de déclaration et d'utilisation d'une méthode.

Exemple 1.26. methode.go

```
package main

import "fmt"

type cuple struct {
 first  int
 second int
}

func (c cuple) add() int {
 return c.first + c.second
}

func main() {
 var c cuple = cuple{first: 1, second: 2}
```

```
 fmt.Println(c.add())
}
```

Dans ce programme on définit une méthode add qui s'applique sur une variable de type cuple, sans prendre de paramètres supplémentaires, et qui retourne un entier (qui est la somme des deux champs de la variable sur laquelle on applique la méthode).

TODO Exercice 2.2.0 : Récupérez le fichier methode.go et testez-le.

Une méthode peut bien sûr modifier la valeur de la variable à laquelle elle s'applique. Cependant, comme d'habitude, pour cela il faut utiliser un pointeur (le passage d'arguments se fait toujours par valeur). Le programme methode2.go donne un exemple de cela.

Exemple 1.27. methode2.go

```
package main

import "fmt"

type cuple struct {
 first  int
 second int
}

func (c *cuple) exchange() {
 c.first, c.second = c.second, c.first
}

func main() {
 var c cuple = cuple{first: 1, second: 2}
 fmt.Println(c)
 c.exchange()
 fmt.Println(c)
}
```

ans ce programme on définit une méthode exchange qui échange les valeurs des deux champs (first et second) d'une variable de type cuple.

TODO Exercice 2.2.1 : Récupérez le fichier methode2.go et testez-le. Que remarquez-vous sur la manière dont on appelle la méthode exchange?

On ne peut définir des méthodes que sur des types qui sont définit au sein du même paquet. Ainsi, vous ne pouvez par exemple pas définir une méthode sur des entiers (int) mais vous pouvez redéfinir un type qui correspond aux entiers et une méthode sur ce nouveau type.

1.6.1.2.3. Retour sur les tableaux

TODO Exercice 2.3.0 : Écrivez une fonction qui prend en argument un tableau d'entiers et ajoute 1 à chaque valeur contenue dans celui-ci. Testez votre fonction sur plusieurs tableaux.

La fonction que vous avez écrite prend en argument un tableau d'entiers et pas un pointeur vers un tableau d'entiers. Pourtant, les valeurs contenues dans le tableau sont bien modifiées. Cela veut dire que le passage de l'argument s'est fait par référence. Nous allons maintenant essayer de bien comprendre cela.

1.6.1.2.3.1. Tableaux, slices et pointeurs

En réalité, en Go, il existe deux types de tableaux. Nous utiliserons leurs noms anglais pour les distinguer : array et slice.

1.6.1.2.3.1.1. Array

Un array est un tableau de taille fixe, sa taille faisant partie intégrale de son type. Ainsi, nous avons des tableaux d'entiers de longueur 1, 2, 3, etc, qui sont des types différents, que l'on note [1]int, [2]int, [3]int, etc. Un tel tableau de taille n correspond juste à n espaces mémoire contigus de la taille d'un élément du tableau. Ces tableaux sont des valeurs normales et sont donc passés par valeur aux fonctions, comme on peut le voir dans le programme array.go.

Exemple 1.28. array.go

```
package main

import "fmt"

func f(t [4]int) {
 for i := 0; i < len(t); i++ {
  t[i] = 0
 }
}

func main() {
 var tab [4]int = [4]int{1, 2, 3, 4}
 fmt.Println(tab)
 f(tab)
 fmt.Println(tab)
}
```

TODO Exercice 2.3.1 : Récupérez le fichier array.go et exécutez-le. Constatez que le tableau tab n’est pas modifié par la fonction f. Modifiez le code en utilisant un pointeur pour faire en sorte que la fonction f modifie tab.

TODO Exercice 2.3.2 : Concevez un programme permettant de savoir quelle est la valeur par défaut d’une variable de type [5]int, d’une variable de type [3]string et d’une variable de type [1]bool. Testez-le.

1.6.1.2.3.1.2. Slice

Les tableaux que nous avons utilisés dans le précédent sujet sont des tableaux de taille variable, que l'on appelle slice en anglais. Il n’existe qu’un seul type de slice d’entiers, que l'on note []int. Un slice est constitué de trois éléments : un entier indiquant sa taille, un entier indiquant sa capacité (nous reviendrons là dessus un peu plus tard) et un pointeur vers un array. Quand on accède à une case d’un slice c’est donc légèrement différent de quand on accède à une case d’un array : il y a un déréférencement qui a lieu. Ainsi, dans le programme slice.go la fonction f modifie vraiment le tableau tab.

Exemple 1.29. slice.go

```
package main

import "fmt"

func f(t []int) {
 for i := 0; i < len(t); i++ {
  t[i] = 0
 }
}

func main() {
 var tab []int = []int{1, 2, 3, 4}
 fmt.Println(tab)
 f(tab)
 fmt.Println(tab)
}
```

TODO Exercice 2.3.3 : Récupérez le fichier slice.go et exécutez-le. Constatez que le tableau tab est modifié par la fonction f.

Il est assez rare que l'on utilise les array en Go, à partir de maintenant nous nous concentrerons sur les slices, que nous appellerons simplement tableaux.

1.6.1.2.3.2. Découpages

Il est possible de récupérer une partie d’un tableau et de la stocker dans un autre tableau. Ceci se fait en écrivant entre crochets, séparés par le symbole :, les indices de début et de fin (exclu) de la partie de tableau qu’on souhaite couper. Le programme slice2.go montre quelque exemples de cette opération.

Exemple 1.30. slice2.go

```
package main

import "fmt"

func main() {
 var tab []int = []int{2, 12, 22, 11, 3, 5, 17, 92, 7}
 fmt.Println(tab)

 var t1 []int = tab[3:7]
 fmt.Println(t1)

 var t2 []int = tab[:7]
 fmt.Println(t2)

 var t3 []int = tab[3:]
 fmt.Println(t3)
}
```

TODO Exercice 2.3.4 : Récupérez le fichier slice2.go et exécutez-le. Que se passe-t-il quand on ne met pas d'indice de début? Que se passe-t-il quand on ne met pas d'indice de fin ?

Exemple 1.31. slice3.go

```
package main

import "fmt"

func main() {
 var tab []int = []int{2, 12, 22, 11, 3, 5, 17, 92, 7}
 var t []int = tab[3:7]
 fmt.Println(tab)
 for i := 0; i < len(t); i++ {
  t[i] = 0
 }
 fmt.Println(tab)
}
```

TODO Exercice 2.3.5 Récupérez le fichier slice3.go sur et exécutez-le. Expliquez à l'aide de l'analogie des boîtes ce que vous observez.

Important

Attention, quand on découpe des tableaux le résultat qu'on obtient est stocké en mémoire au même endroit que le tableau de départ (plus précisément, les pointeurs des deux tableaux indiquent le même tableau — array — sous-jacent). Si l'on modifie l'un, cela modifie aussi l'autre.

1.6.1.2.3.3. Un peu plus sur les chaînes de caractères

Les chaînes de caractères (string) se manipulent comme des tableaux. On peut accéder à un caractère d'une chaîne s par son indice, par exemple s[2] pour le troisième caractère. On peut donc aussi découper les chaînes de caractères avec la syntaxe vue juste au dessus.

TODO Exercice 2.3.6 : Écrivez une fonction qui prend en argument une chaîne de caractères constituée de deux mots séparés par un espace et retourne deux chaînes de caractères, chacune contenant l'un des deux mots. Afin de vérifier si un caractère d'une chaîne est un espace on peut le comparer à ' ', par exemple s[2] == ' ' vaut true si le troisième caractère de s est un espace et false sinon. Testez votre fonction sur plusieurs exemples.

1.6.1.2.3.4. Création de tableaux

Jusqu'à présent, on a toujours créé nos tableaux en indiquant directement leur contenu. Cependant, on peut parfois vouloir simplement créer un tableau d'une taille donnée, sans pour autant spécifier ce qui va dedans, par exemple

quand on va remplir ce tableau par la suite en fonction de résultats de calculs. Le programme slice4.go montre quelques exemples.

Exemple 1.32. slice4.go

```
package main

import "fmt"

func main() {
 var itab []int = make([]int, 5)
 var btab []bool = make([]bool, 3)
 fmt.Println(itab, btab)
}
```

Dans ce programme on utilise make, qui prend deux arguments, pour créer nos tableaux. Le premier argument que l'on doit donner est le type du tableau à créer ([]int, []bool, []string, etc). Le deuxième argument est un entier qui sert à indiquer la taille du tableaux que l'on veut créer.

TODO Exercice 2.3.7 : Récupérez le fichier slice4.go sur et exécutez-le. Quelles sont les valeurs contenues dans un tableau créé à l'aide de make ?

TODO Exercice 2.3.8 : Écrivez une fonction qui prend en argument un entier n, crée un tableau contenant les n premiers multiples de 7, et retourne ce tableau. Testez-votre fonction pour plusieurs valeurs de n (0, 5, 100, etc).

1.6.1.2.3.5. Agrandissement et capacité

On a appelé les slices tableaux de taille variable, mais, pour le moment, on n'a vu que le moyen de réduire leur taille en les découpant. Il est aussi possible d'augmenter la taille d'un tableau. Ceci se fait en utilisant la fonction append.

1.6.1.2.3.5.1. La fonction append

Cette fonction prend en arguments un tableau et un élément et retourne un tableau correspondant au tableau donné en argument à la suite duquel l'élément donné en argument a été ajouté. Le programme Section 1.6.1.2.3.5.2, « La capacité » montre un exemple.

Exemple 1.33. slide5.go

```
package main

import "fmt"

func main() {
 var tab []int = make([]int, 5)
 fmt.Println(tab, len(tab))
 tab = append(tab, 2)
 fmt.Println(tab, len(tab))
}
```

TODO Exercice 2.3.9 : Récupérez le fichier slice5.go et exécutez-le.

1.6.1.2.3.5.2. La capacité

On a indiqué précédemment qu'un tableau possède, en plus de sa taille, une capacité. Celle-ci correspond à une quantité d'espace réservée pour stocker le tableau : la taille du tableau est son nombre d'éléments, sa capacité est son nombre d'éléments potentiels. On peut voir la capacité comme la taille de l'array sous-jacent. Elle peut se définir en ajoutant un troisième argument à make lorsque l'on crée un tableau, et on peut la connaître en utilisant la fonction cap. Le programme ??? montre quelques exemples.

Exemple 1.34. slice6.go

```
package main
```

```
import "fmt"

func main() {
 var tab []int = make([]int, 3, 5)
 fmt.Println(tab, len(tab), cap(tab))

 var tab2 []int = make([]int, 3)
 fmt.Println(tab2, len(tab2), cap(tab2))

 var tab3 []int = []int{0, 0, 0}
 fmt.Println(tab3, len(tab3), cap(tab3))
}
```

TODO Exercice 2.3.10 : Récupérez le fichier slice6.go et exécutez-le. Quelle est la capacité d'un tableau créé par make lorsque l'on n'indique que la taille de ce tableau ? Quelle est-elle lorsque l'on crée le tableau sans utiliser make?

1.6.1.2.3.5.3. Lien entre append et capacité

Lorsque l'on utilise append le résultat va varier en fonction de la capacité : si celle-ci est suffisante, l'élément sera ajouté dans l'array sous-jacent, c'est-à-dire au même endroit dans la mémoire que le tableau d'origine. Par contre, si la capacité est insuffisante, le tableau d'origine sera copié dans un espace plus grand, puis l'élément sera ajouté à la suite. Le programme slice7.go permet de mieux comprendre cela.

Exemple 1.35. slide7.go

```
package main

import "fmt"

func main() {
 var length int = 3
 var capacity int = 5
 var tab []int = make([]int, length, capacity)
 var t []int = tab
 var max int = 5
 for i := length; i <= max; i++ {
  t = append(t, i)
  t[0] = i
  fmt.Println("Tour de boucle", i)
  fmt.Println("\ttab =", tab, "len(tab) =", len(tab), "cap(tab) =", cap(tab))
  fmt.Println("\tt =", t, "len(t) =", len(t), "cap(t) =", cap(t))
 }
}
```

TODO Exercice 2.3.11 : Récupérez le fichier slice7.go et exécutez-le.

Au début, on peut observer que lorsque l'on modifie la première case de t cela change aussi le contenu de la première case de tab. En effet, ces deux tableaux correspondent au même espace en mémoire (ils ont le même tableau sous-jacent). On remarque ensuite que, au tour de boucle 4, la taille de t devient égale à sa capacité. Au tour suivant la capacité de t a augmenté : le tableau a été copié dans un espace plus grand. On voit que c'est bien une copie car à partir de ce moment, modifier la première case de t n'affecte plus la première case de tab.

TODO Exercice 2.3.12 : Testez à nouveau le programme du fichier slice7.go en modifiant les valeurs de length, capacity et max. Essayez d'en déduire une règle sur la façon dont la capacité d'un tableau est augmentée lorsque c'est nécessaire.

Le fait de copier un tableau a un coût, c'est pour essayer d'éviter au maximum les copies quand on ajoute des éléments qu'on permet de réserver un espace mémoire plus grand que nécessaire pour nos tableaux (grâce à la capacité).

1.6.1.2.3.5.4. Lien entre découpages et capacité

Lorsque l'on découpe un tableau, comme on l'a vu plus haut, on peut se demander quelle sera sa capacité. Ceci peut se déterminer expérimentalement.

TODO Exercice 2.3.13 Récupérez le fichier slice8.go sur. Exécutez-le pour différentes valeurs des variables length, capacity, start et end et déduisez-en comment sont calculées la taille et la capacité d'un découpage de tableau. Testez aussi des valeurs de end qui dépasse la taille ou même la capacité de tab.

Exemple 1.36. slice8.go

```
package main

import "fmt"

func main() {
 var length int = 5
 var capacity int = 10
 var tab []int = make([]int, length, capacity)

 var start int = 0
 var end int = 5
 var t []int = tab[start:end]

 fmt.Println(len(tab), cap(tab))
 fmt.Println(len(t), cap(t))
}
```

1.6.1.2.3.6. Copies de tableaux

On l'a vu dans les parties précédentes : les tableaux peuvent s'avérer complexes et il est parfois difficile de savoir si deux tableaux partagent un même espace mémoire (et donc si modifier l'un affectera l'autre) ou pas.

Lorsqu'on a besoin de récupérer tout ou partie d'un tableau dans un autre, en s'assurant que les deux tableaux sont indépendants, on peut utiliser la fonction copy. Le programme slice9.go donne un exemple d'utilisation de cette fonction.

Exemple 1.37. slice9.go

```
package main

import "fmt"

func main() {
 var tab []int = []int{1}
 var t []int = make([]int, 1)
 copy(t, tab)
 fmt.Println("tab:", tab, len(tab), cap(tab))
 fmt.Println("t:", t, len(t), cap(t))

 t[0] = 0
 fmt.Println("tab:", tab, len(tab), cap(tab))
 fmt.Println("t:", t, len(t), cap(t))
}
```

On peut observer que la fonction copy s'appelle avec deux arguments. Le premier est la destination : l'endroit où on va placer la copie. Le deuxième est la source : le tableau qui doit être copié.

TODO Exercice 2.3.14 : Récupérez le fichier slice9.go sur et exécutez-le.

En testant, on se rend compte que lorsque l'on modifie la première case du tableau t cela n'a pas d'effet sur le tableau tab. On à bien réalisé une copie.

TODO Exercice 2.3.15 : Dans le fichier slice9.go faites varier la taille et la capacité du tableau t et la taille du tableau tab. Que se passe-t-il quand la taille de t est inférieure à celle de tab? Que se passe-t-il quand la capacité de t est inférieure à la taille de tab ?

1.6.1.2.3.7. Tableaux de tableaux (de tableaux de tableaux. . .)

Puisque l'on peut créer un tableau d'éléments de n'importe quel type, on peut aussi créer des tableaux de tableaux. Le programme tabtab.go donne un exemple.

Exemple 1.38. tabtab.go

```
package main

import "fmt"

func main() {
 var tab [][]int = [][]int{
  []int{1, 2, 3},
  []int{4, 5},
  []int{6, 7, 8, 9, 10},
 }

 fmt.Println("tab =", tab)
 fmt.Println("tab[0] =", tab[0])
 fmt.Println("tab[0][1] =", tab[0][1])
}
```

Dans ce programme on observe tout d'abord la création d'un tableau de tableaux d'entiers. Ici on dit explicitement le contenu de notre tableau. Son type est [][]int donc sont contenu est écrit sous la forme [][]int{contenu}. Les éléments qu'il contient sont des tableaux d'entiers (type []int) donc chacun d'entre eux est déclaré exactement comme on l'a vu avant : []int{contenu}.

Ensuite, on voit comment accéder aux éléments de ce tableau : tab fait référence à tout le tableau, tab[i] fait référence au tableau d'entiers numéro i dans le tableau de tableaux, et tab[i][j] fait référence à l'entier numéro j de ce tableau numéro i.

TODO Exercice 2.3.16 : Récupérez le fichier tabtab.go et exécutez-le.

On peut bien sûr utiliser make pour créer des tableaux de tableaux. Il faudra alors bien penser à initialiser (par exemple avec make), si nécessaire, chacun des tableaux de notre tableau de tableaux.

TODO Exercice 2.3.17 : Écrivez un programme qui initialise un tableau t de n tableaux de taille m, puis qui le rempli de telle sorte que t[i][j] soit égal à i × j. On construit donc un tableau contenant les tables de multiplication de 0 à n-1, chacune rangée dans un tableau. Testez votre programme pour différentes valeurs de n et m.

Vous pouvez trouver une très bonne explication, plus détaillée, du fonctionnement interne des tableaux en Go sur le site go.dev[7]

1.6.1.2.4. Arguments de la ligne de commande

Dans les exercices précédents on a souvent testé des programmes pour différentes valeurs de variables. Pour cela, il fallait à chaque fois modifier le code du programme puis utiliser la commande go run (qui elle-même compile le fichier et l'exécute). Ce n'est pas forcément raisonnable de faire cela : modifier le code à chaque fois est source d'erreurs et la compilation a un coût (en énergie et en temps, faible pour les programmes que vous avez écrits jusqu'à présent, mais qui peut être beaucoup plus élevé dès que les programmes deviennent plus complexes).

Nous allons voir dans cette partie comment il est possible de modifier les valeurs de certaines variables au moment de l'exécution d'un programme : sans avoir à la recompiler et sans avoir à modifier le code source.

1.6.1.2.4.1. Séparer la compilation de l'exécution

Quand on exécute un programme prog.go en utilisant la commande go run prog.go cela donne en réalité lieu (de manière transparente) à deux étapes : la compilation, qui permet de produire un fichier exécutable à partir du fichier source, puis l'exécution qui permet de faire fonctionner le programme.

Parfois on souhaite exécuter un programme plusieurs fois, sans avoir modifié son code. Il n'est alors pas raisonnable de le recompiler à chaque fois. On peut séparer la compilation de l'exécution en utilisant la commande go build à la place de go run.

TODO Exercice 2.4.0 : Compilez le programme tabtab.go en utilisant la commande go build tabtab.go à la place de go run tabtab.go. Vous obtenez normalement un fichier exécutable nommé tabtab.

[7] https://go.dev/blog/slices-intro

Le fichier exécutable obtenu par la compilation peut ensuite être utilisé en tapant dans le terminal un chemin vers ce fichier. Si vous vous situez dans le répertoire où est le fichier exécutable prog vous pouvez utiliser la commande ./prog pour l'exécuter.

TODO Exercice 2.4.1 : Exécutez le programme tabtab créé précédemment et vérifiez que vous obtenez bien le même résultat qu'en faisant directement go run tabtab.go.

1.6.1.2.4.2. Définir des valeurs à l'exécution

Lorsque vous utilisez une commande dans un terminal, vous indiquez d'abord le nom de cette commande, puis un certain nombre d'arguments séparés par des espaces. Le résultat de la compilation d'un programme Go est une commande, vous pouvez donc lui donner des arguments. Ainsi, si j'ai construit un exécutable prog je peux utiliser la commande ./prog 12 bonjour pour lui donner les arguments 12 et bonjour. Il est ensuite possible pour le programme de récupérer et d'utiliser ces arguments. Pour cela on utilise la bibliothèque os (qu'il faudra penser à importer).

Le programme args.go donne un exemple de récupération d'arguments dans un programme Go à l'aide de la bibliothèque os.

Exemple 1.39. args.go

```
package main

import (
 "fmt"
 "os"
)

func main() {
 fmt.Println("Le programme s'appelle", os.Args[0])
 fmt.Println("Il y a", len(os.Args)-1, "arguments")
 for argPos := 1; argPos < len(os.Args); argPos++ {
  fmt.Println("L'argument", argPos, "vaut", os.Args[argPos])
 }
}
```

Une fois la bibliothèque os importée on a accès à un tableau os.Args qui contient le nom du pro- gramme exécuté, suivi des arguments donnés à ce programme. On peut donc obtenir le nombre d'arguments en regardant la longueur de ce tableau. Et on peut accéder aux arguments dans les cases du tableau.

On notera que l'importation de plusieurs bibliothèque peut se faire en les listant toutes, une par ligne, entre parenthèse après le import.

TODO Exercice 2.4.2 : Récupérez le fichier args.go et compilez-le avec go build. Testez le fichier exécutable obtenu (args) avec différents nombres et types d'arguments.

TODO Exercice 2.4.3 : Exécutez le fichier args.go avec go run. Où se trouve le programme compilé par cette commande ?

TODO Exercice 2.4.4 : Écrivez un programme qui stocke son premier argument dans une variable entière. Compilez- le avec go build. Que pouvez-vous déduire sur la nature des arguments récupérés par un programme ?

On dispose d'un certain nombre de fonctions utiles pour transformer des chaînes de caractères en d'autres types. Nous ne les verrons pas toutes ici, mais pour interpréter une chaîne de caractères comme un entier on peut utiliser strconv.Atoi qui provient de la bibliothèque strconv. Cette fonction prend en argument une chaîne de caractère et retourne un entier et une erreur. L'erreur sert à savoir s'il y a eu un problème durant la conversion (si elle vaut nil c'est que tout s'est bien passé). L'entier est l'interprétation de la chaîne de caractères sous forme d'entier. Pour plus de précisions, n'hésitez pas à consulter la documentation.

TODO Exercice 2.4.5 Modifiez votre programme précédent pour convertir votre argument en entier avant de le stocker dans votre variable (n'oubliez pas de récupérer l'erreur dans une autre variable). Compilez à nouveau votre programme (go build). Testez-le ensuite pour différentes valeurs d'argument (y compris des arguments qui ne sont pas des entiers), ainsi que sans argument. Analysez les résultats et les erreurs obtenues.

Tout ceci est sans doute un peu laborieux pour récupérer des arguments. Il existe une bibliothèque pour vous simplifier la vie : la bibliothèque flag.

1.6.1.2.4.3. La bibliothèque flag

La bibliothèque flag peut se charger pour vous de mettre les arguments d'un programme dans des variables, en s'assurant qu'ils ont le bon type et même en leur donnant une valeur par défaut. Le programme flag.go donne un exemple d'utilisation de cette bibliothèque.

Exemple 1.40. flag.go

```
package main

import (
 "flag"
 "fmt"
)

func main() {
 var n int
 flag.IntVar(&n, "setn", 27, "Fixe la valeur de n (défaut : 27)")
 flag.Parse()
 fmt.Println("n vaut", n)
}
```

Dans ce programme on commence par déclarer un entier n. Ensuite on utilise la fonction flag.IntVar qui prépare la lecture d'un argument de type entier. Cet argument sera stocké dans la variable n (on donne pour cela un pointeur vers n en premier argument à la fonction). Il aura pour valeur par défaut 27 (troisième argument de la fonction). Dans la ligne de commande, pour fixer une valeur à cet argument il faudra écrire -setn=12 (on peut remplacer 12 par n'importe quelle autre valeur).

Enfin, la fonction flag.Parse() permet de prendre en compte toutes les lectures d'arguments qui ont été préparées avant (ici par flag.IntVar).

TODO Exercice 2.4.6 : Récupérez le fichier flag.go et compilez-le avec go build. Essayez ensuite de l'exécuter de différentes façon (avec et sans argument, avec différents arguments).

TODO Exercice 2.4.7 Testez votre programme avec l'argument -h, c'est-à-dire en faisant ./flag -h. À quoi sert le quatrième argument de la fonction flag.IntVar ?

On peut bien sûr traiter d'autres types d'arguments que des entiers (avec les fonctions flag.XxxVar adéquates), plusieurs arguments à la fois (avec plusieurs fonctions flag.XxxVar avant le flag.Parse), etc. Toutes les informations sur la bilbiothèque sont disponibles dans la documentation[8].

TODO Exercice 2.4.8 : Écrivez un programme qui prend en argument trois entiers x, y et z et stocke dans un tableau la table de multiplication de x à partir de 0 × x jusqu'à y × x à part les valeurs qui sont multiples de z. Testez votre programme pour plusieurs valeurs de x, y et z (sans le recompiler). Si nécessaire, modifiez votre programme pour vous assurer que la taille de votre tableau est juste la bonne pour stocker vos valeurs. Vous devez être capable de justifier que c'est bien le cas. Testez-le. Si nécessaire, modifiez votre programme pour vous assurer qu'il n'y a jamais de copies de votre tableau qui sont réalisées au cours de son remplissage. À nouveau, vous devez être capable de justifier que c'est bien le cas. Testez-le.

1.6.1.3. Partie 3 : fichiers

1.6.1.3.1. Écriture de fichiers

Il est pratique de savoir écrire des données dans des fichiers. Ceci peut permettre de sauvegarder l'état d'un programme entre plusieurs utilisations de celui-ci, ou encore d'enregistrer les données de sortie d'un programme pour ensuite les analyser à l'aide d'un autre programme. Nous allons ici nous concentrer sur la création d'un fichier dans lequel on écrira ensuite, mais il est bien sûr possible d'ouvrir un fichier déjà existant pour écrire dedans (soit après son contenu, soit en effaçant celui-ci, voir même au milieu du fichier).

[8] https://pkg.go.dev/flag

1.6.1.3.2. Créer un fichier

Pour créer un fichier on peut utiliser la fonction Create du paquet os[9]. Il ne faut pas oublier, une fois que l'on a terminé d'utiliser le fichier, de le fermer avec la fonction Close. Le programme 1 donne un exemple de création de fichier.

Exemple 1.41. create.go

```
package main

import (
 "log"
 "os"
)

func main() {
 var myFile *os.File
 var err error
 myFile, err = os.Create("monfichier")
 if err != nil {
  log.Fatal(err)
 }
 err = myFile.Close()
 if err != nil {
  log.Fatal(err)
 }
}
```

La fonction Create prend en argument le nom du fichier à créer. Elle retourne le fichier lui-même (myFile) ainsi qu'une erreur (err). Celle-ci sert à indiquer si jamais il y a eu un soucis en créant le fichier. On doit donc toujours vérifier que ce n'est pas le cas (en vérifiant que l'erreur vaut nil) avant d'utiliser le fichier.

La fonction Close retourne elle aussi une erreur, pour indiquer si le fichier a pu être fermé correctement ou pas. Par ailleurs, vous aurez peut-être noté que cette fonction ne s'utilise pas exactement comme les autres fonctions que vous avez vues jusqu'à présent : on écrit myFile.Close() plutôt que os.Close(myFile). On appelle ce genre de fonction une méthode et vous verrez un peu plus tard comment en définir soit même et à quoi elles servent.

On notera aussi l'utilisation du paquet log[10] pour afficher des informations sur le déroulement du programme à l'écran (dans les TPs précédents on utilisait fmt). En particulier, la fonction Fatal de ce paquet permet de terminer le programme en plus d'afficher des informations à l'écran.

TODO Exercice 3.1.0 : Récupérez le programme create.go et testez-le. Constatez que celui-ci crée effec- tivement des nouveaux fichiers. Que se passe-t-il si on demande de créer un fichier qui existe déjà ?

1.6.1.3.3. Écrire dans un fichier nouvellement créé

Une fois notre fichier créé on peut écrire dedans avec la fonction Fprintln du paquet fmt. Cette fonction est similaire à Println que nous avons déjà vue, à part qu'elle écrit dans un fichier, qui doit être indiqué dans ses arguments. Le programme write.go donne un exemple d'utilisation de cette fonction.

Exemple 1.42. write.go

```
package main

import (
 "fmt"
 "log"
 "os"
)

func main() {
 var myFile *os.File
```

[9] https://pkg.go.dev/os
[10] https://pkg.go.dev/log

```
 var err error
 myFile, err = os.Create("monfichier")
 if err != nil {
  log.Fatal(err)
 }

 _, err = fmt.Fprintln(myFile, "Bonjour")
 if err != nil {
  log.Fatal(err)
 }

 _, err = fmt.Fprintln(myFile, "Au revoir")
 if err != nil {
  log.Fatal(err)
 }

 err = myFile.Close()
 if err != nil {
  log.Fatal(err)
 }
}
```

Ce programme est similaire au programme create.go à la différence qu'entre la création et la fermeture de fichier on utilise la fonction Fprintln. On peut noter que le fichier dans lequel écrire est donné en premier argument de la fonction. Par ailleurs, cette fonction peut retourner une erreur si l'écriture s'est mal passée (ici on la récupère dans err et on vérifie que tout va bien en la comparant à nil). Une autre valeur est retournée, qu'on ne récupère pas ici (on utilise le symbole _ pour cela, car on ne souhaite pas utiliser cette valeur et, comme nous l'avons vu précédemment, toute variable créée en Go doit ensuite être utilisée). Cette valeur indique le nombre d'octets qui ont été écrits dans le fichier.*La fonction Println que nous avons utilisée auparavant retourne en fait elle aussi deux valeurs : le nombre d'octets écrits et une erreur.*

TODO Exercice 3.1.1 : Récupérez le programme write.go et exécutez-le. Essayez de modifier différents éléments de ce programme (nom du fichier, contenu des lignes écrites, nombre de lignes écrites) et testez à nouveau. Que se passe-t-il si on utilise ce programme avec un nom de fichier correspondant à un fichier déjà existant et contenant du texte ?

TODO Exercice 3.1.2 : Écrivez un programme qui prend en argument un entier n compris entre 0 et 10 (il faudra signaler si l'entier n'est pas correct) et écrit la table de multiplication de n, bien présentée, dans un fichier (de 0 × n à 10 × n).

1.6.1.3.4. Lecture de fichiers

Il est aussi utile d'être capable de lire un fichier et de traiter son contenu. Ceci peut servir en particulier à configurer un logiciel : on crée un fichier de configuration qui contient des informations sur nos préférences (langue, raccourcis, etc) et la première action du logiciel lorsqu'il démarre est de lire ce fichier pour prendre en compte ces préférences. Une autre utilisation courante est de stocker un ensemble de données sous forme d'un fichier (plus ou moins structuré : csv, json, etc) pour pouvoir ensuite les traiter à l'aide d'un programme en lisant le fichier.

1.6.1.3.4.1. Lire tout un fichier d'un coup

La méthode la plus simple en Go pour récupérer le contenu d'un fichier est d'utiliser la fonction ReadFile du paquet io/ioutil [11]. Cette fonction permet de lire l'intégralité d'un fichier (identifié par son chemin) et de stocker ceci dans un tableau d'octets ([]byte). Les raisons pour lesquelles on obtient un tableau d'octets plutôt que directement une chaîne de caractères sont en dehors du cadre de ce cours (mais vous en parlerez dans vos cours d'architecture et de système). Pour l'instant il vous suffit de vous souvenir qu'on peut transformer ce tableau d'octets t en chaîne de caractères simplement en écrivant string(t). Le programme 3 donne un exemple d'utilisation de cette fonction.

Exemple 1.43. readfile.go

```
package main

import (
```

[11] https://pkg.go.dev/io/ioutil

```
 "io/ioutil"
 "log"
)

func main() {
 var filePath string = "test"
 var data []byte
 var err error
 data, err = ioutil.ReadFile(filePath)
 if err != nil {
  log.Fatal(err)
 }
 log.Print("J'ai lu : ", string(data))
}
```

Dans ce programme on remarque que la fonction ReadFile retourne deux valeurs : le résultat de la lecture et une erreur. Avant de consulter le résultat de la lecture on doit toujours vérifier que l'erreur vaut nil, indiquant que tout s'est bien passé.

TODO Exercice 3.2.0 : Récupérez le programme readfile.go et testez-le. Expliquez l'erreur que vous observez. Créez au bon endroit (pour qu'il n'y ait pas d'erreur) un fichier test contenant quelques lignes de texte puis testez à nouveau votre programme.

Une fois le fichier lu et stocké sous forme de chaîne de caractères, on peut le traiter en utilisant des fonctions d'analyse des chaînes de caractères, comme celles du paquet strings [12] par exemple.

Pour des raisons de performances, il n'est pas toujours raisonnable de lire un fichier entier (et donc de le stocker en mémoire). Dans la suite de cette partie nous allons voir qu'il est possible de lire un fichier morceau par morceau.

1.6.1.3.4.2. Extraire des données d'un fichier

En utilisant le paquet os on a accès aux fonctions de bas niveau permettant de lire un fichier octet par octet (notamment la fonction Read du paquet os). Ces fonctions peuvent être un peu compliquées à manipuler (pour des raisons techniques, c'est parfois difficile de reconstituer une ligne de texte à partir de celles-ci) cependant on a aussi accès à des fonctions de plus haut niveau, construite à partir des fonctions du paquet os, permettant de lire plus simplement des fichiers.

Le paquet fmt [13], par exemple, fournit quelques fonctions intéressantes. Nous allons étudier en particulier la fonction Fscanln qui permet de lire une ligne d'un fichier et de stocker les différentes valeurs stockées sur cette ligne (séparées par des espaces) dans des variables. Le programme fscanln.go montre un exemple d'utilisation de cette fonction.

Exemple 1.44. fscanln.go

```
package main

import (
 "fmt"
 "log"
 "os"
)

func main() {
 // Ouverture du fichier
 var filePath string = "fichierScan"
 var myFile *os.File
 var err error
 myFile, err = os.Open(filePath)
 if err != nil {
  log.Fatal(err)
 }
```

[12] https://pkg.go.dev/strings
[13] https://pkg.go.dev/fmt

```
 // Lecture d'une ligne
 var nbLus int
 var unEntier int
 var uneChaine string
 nbLus, err = fmt.Fscanln(myFile, &unEntier, &uneChaine)
 if err != nil {
  log.Fatal(err)
 }
 log.Print("J'ai lu ", nbLus, " valeurs.")
 log.Print("unEntier = ", unEntier)
 log.Print("uneChaine = ", uneChaine)

 // Fermeture du fichier
 err = myFile.Close()
 if err != nil {
  log.Fatal(err)
 }
}
```

La première partie de ce programme consiste à ouvrir un fichier. Pour cela on utilise la fonction Open du paquet os en indiquant le chemin du fichier à ouvrir. Cette fonction retourne un fichier et une erreur. Comme pour ReadFile dans la partie précédente, il faut vérifier que l'erreur vaut nil avant d'utiliser le fichier.

La deuxième partie du programme est la lecture à proprement parler. On commence par préparer une variable nbLus qui servira à indiquer combien de valeurs ont été lues avec succès ainsi que deux variables unEntier et uneChaine pour stocker les données que l'on souhaite extraire du fichier. On utilise ensuite la fonction Fscanln en lui donnant en arguments le fichier à lire (ici myFile) et les adresses des variables où stocker les données. Cette fonction peut retourner une erreur si les choses se sont mal passées, on doit vérifier que cette erreur est différente de nil avant d'utiliser les données récupérées.

La troisième partie du programme ferme le fichier.

TODO Exercice 3.2.1 : Pourquoi utilise-t-on les adresses des variables unEntier et uneChaine plutôt que directement les variables elles-mêmes ?

TODO Exercice 3.2.2 : Récupérez le programme fscanln.go et exécutez-le. Expliquez l'erreur que vous rencontrez. Créez ensuite un fichier fichierScan et testez à nouveau. Expliquez l'erreur que vous rencontrez. Ajoutez une ligne quelconque au fichier fichierScan et testez à nouveau. Expliquez l'erreur que vous rencontrez. Changez fichierScan pour qu'il ne contienne que des lignes avec un entier suivi d'un espace suivi d'une chaîne de caractères sans espaces. Testez à nouveau.

Après qu'une ligne ait été lue, la position actuelle dans le fichier est mise à jour dans la structure de données myFile. Le prochain appel à Fscanln lira donc la ligne suivante.

TODO Exercice 3.2.3 : Modifiez le programme fscanln.go pour qu'il lise 5 lignes du fichier fichierScan à l'aide d'une boucle. Testez-le. Corrigez si nécessaire le contenu du fichier fichierScan pour que le pro- gramme ne produise plus d'erreur à la lecture de ce fichier. Testez à nouveau.

Lorsque la fonction Fscanln atteint la fin du fichier lu, elle retourne une erreur particulière (que vous avez peut être observée dans vos tests). Cette erreur s'appelle EOF et elle fait partie du paquet io. On peut donc y faire référence dans du code Go en écrivant io.EOF.

TODO Exercice 3.2.4 : Modifiez le programme fscanln.go pour qu'il lise le fichier fichierScan jusqu'au bout, quelque soit sa taille. Vous pourrez utilisez pour cela une conditionnelle dans votre boucle, qui teste si l'erreur est égale à io.EOF et stoppe la boucle si c'est le cas.*De la même façon que Fprintln correspond à Println, Fscanln correspond à une fonction Scanln qui permet de lire ce que l'utilisateur entre au clavier dans le terminal.*

1.6.1.3.4.3. Lire un fichier ligne par ligne

La fonction Fscanln n'est pas toujours la plus pratique à utiliser, en particulier car elle impose que les données soient séparées par des espaces. Nous allons maintenant voir un autre moyen de lire les données d'un fichier, en utilisant des fonctions du paquet bufio [14].*Les fonctions du paquet bufio sont des entrées/sorties bufferisées, c'est*

[14] https://pkg.go.dev/bufio

à dire que lorsque l'on demande une lecture les données sont lues dans un espace mémoire (un tampon, buffer en anglais) où elles ont été préalablement préchargées depuis le fichier. Quand l'espace est vide, des données sont rechargées dedans, jusqu'à le remplir. Ceci limite les accès aux fichiers, qui sont des opérations coûteuses pour un ordinateur.

Le programme bufio.go montre comment préparer un fichier à être lu, puis comment lire quelques lignes de celui-ci.

Exemple 1.45. bufio.go

```
package main

import (
 "bufio"
 "log"
 "os"
)

func main() {
 // Ouverture du fichier
 var filePath string = "test"
 var myFile *os.File
 var err error
 myFile, err = os.Open(filePath)
 if err != nil {
  log.Fatal(err)
 }

 // Préparation de la lecture
 var scanner *bufio.Scanner
 scanner = bufio.NewScanner(myFile)

 // Lecture des lignes du fichier
 for scanner.Scan() {
  log.Print("Je viens de lire : ", scanner.Text())
 }

 // Vérification que tout s'est bien passé
 if scanner.Err() != nil {
  log.Fatal(scanner.Err())
 }

 // Fermeture du fichier
 err = myFile.Close()
 if err != nil {
  log.Fatal(err)
 }
}
```

La première partie de ce programme consiste à ouvrir un fichier, exactement comme nous l'avons vu avant.

La deuxième partie du programme utilise la fonction NewScanner du paquet bufio pour construire un scanner qui sera utilisé pour lire le fichier.

La troisième partie utilise la méthode Scan pour lire, en boucle, le fichier ligne par ligne. À chaque tour de boucle on récupère la ligne de texte lue grâce à la méthode Text. La boucle s'arrête quand toutes les lignes ont été lues ou quand il y a un problème à la lecture : dans ces deux cas la méthode Scan retourne false.

Enfin, en utilisant la méthode Err, on vérifie que la lecture s'est bien passée et que l'on a bien atteint la fin du fichier.

On peut noter que tout ceci fonctionne bien car scanner est un pointeur vers un bufio.Scanner, les méthodes et fonctions peuvent donc modifier son contenu (l'endroit où on est dans le fichier, le contenu de la dernière ligne lue, l'erreur).

TODO Exercice 3.2.5 : Récupérez le programme bufio.go et testez-le.

TODO Exercice 3.2.6 : Modifiez le programme bufio.go pour compter le nombre de lignes du fichier et l'afficher. Testez votre code sur différents fichiers.

TODO Exercice 3.2.7 : Modifiez le programme bufio.go pour afficher uniquement les 5 premiers lignes du fichier. Testez votre code sur différents fichiers. En particulier, considérez le cas de fichiers qui contiennent moins de 5 lignes.

1.6.1.3.4.4. Lire un fichier csv

En guise d'entraînement à l'utilisation des fonctions de lecture de fichier nous allons prendre un exemple. Vous trouverez sur un fichier notes.csv. Celui-ci contient sur chaque ligne des données (toutes les notes reçues par des étudiants au cours d'un semestre), séparées par des virgules : un prénom, une note, un bonus, une note totale (qui est la somme de la note et du bonus). Une ligne représente donc une note reçue par un étudiant, il peut y avoir plusieurs lignes pour le même étudiant (s'il a reçu plusieurs notes).

TODO Exercice 3.2.8 : Écrire un programme qui vérifie que chaque note totale est bien la somme de la note et du bonus dans le fichier notes.csv en affichant le numéro de chaque ligne où ce n'est pas le cas. Vous pourrez utiliser la fonction Split du paquet strings pour découper vos lignes à partir des virgules.

TODO Exercice 3.2.9 : Écrire un programme qui produit un fichier corrigé notes_valides.csv à partir de notes.csv. La note totale peut-être modifiée.

TODO Exercice 3.2.10 : Écrire un programme qui calcule la moyenne (à partir des notes totales, intégrant les bonus) d'un étudiant dont le prénom est donné en argument du programme.*Il existe aussi des bibliothèques permettant de lire et écrire simplement des fichiers structurés (json [15], csv [16] xml [17] par exemple). Ces bibliothèques vont directement retranscrire les contenus des fichiers sous forme de structures de données et écrire les contenus de structures de données sous forme de fichiers.*

1.6.1.4. Partie 4 : interfaces et fonctions "avancées"

1.6.1.4.1. Mise en pratique des structures et méthodes

On suppose que l'on souhaite gérer les notes de différentes promotions d'étudiants, calculer les moyennes des étudiants et les classer. Pour cela on va définir un type pour nos étudiants et un type pour une promo (qui contiendra plusieurs étudiants). On définira ensuite un certain nombre de méthodes sur les étudiants et sur les promos.

TODO Exercice 4.1.0 : Définir un type Etudiant comprenant un nom, un prénom et une liste de notes.

TODO Exercice 4.1.1 : Définir une méthode Moyenne qui s'applique à un étudiant et calcule sa moyenne.

TODO Exercice 4.1.2 : de notes. Définir une méthode AjouteNote qui s'applique à un étudiant et ajoute une note dans sa liste.

TODO Exercice 4.1.3 : Définir un type Promotion comprenant un nom de promo et une liste d'étudiants.

TODO Exercice 4.1.4 : de leur moyenne. Définir une méthode Classer qui s'applique à une promotion et classe les étudiants en fonction.

1.6.1.4.2. Les interfaces

Les interfaces permettent de définir des fonctions génériques, qui vont s'appliquer à plusieurs types d'éléments, à condition que ces types partagent un ensemble de méthodes (qui seront utilisées pour les manipuler).

Nous avons déjà utilisé des interfaces sans le savoir quand nous avons créé des bufio.Reader. Ceux-ci peuvent être créés à partir de n'importe quelle variable d'un type qui met en oeuvre l'interface io.Reader, c'est-à-dire qui a une méthode Read.

Une interface est en fait simplement une liste de signatures de méthodes avec un nom. On dit qu'un type qui possède toutes ces méthodes implante l'interface. Dès qu'une fonction prend en paramètre le type de l'interface

[15] `https://pkg.go.dev/encoding/json`
[16] `https://pkg.go.dev/encoding/csv`
[17] `https://pkg.go.dev/encoding/xml`

on peut utiliser à la place tout type qui implante l'interface. Le programme interface.go montre un exemple de définition et d'implantation d'interface.

Exemple 1.46. interface.go

```
package main

import (
 "fmt"
 "math"
)

// Interface
type Forme interface {
 Perimetre() float64
}

func PlusGrandPerimetre(f1, f2 Forme) bool {
 return f1.Perimetre() > f2.Perimetre()
}

// Implantation pour les carrés
type Carre struct {
 Cote float64
}

func (c Carre) Perimetre() float64 {
 return 4 * c.Cote
}

// Implantation pour les cercles
type Cercle struct {
 Rayon float64
}

func (c Cercle) Perimetre() float64 {
 return 2 * math.Pi * c.Rayon
}

func main() {
 var ca Carre = Carre{Cote: 5}
 var ce Cercle = Cercle{Rayon: 2.5}
 fmt.Println(PlusGrandPerimetre(ca, ce))
}
```

Ce programme définit une interface Forme qui décrit une forme générale avec un périmètre. Sur cette interface, une fonction est définie, permettant de comparer les périmètres de deux formes.

Ensuite, deux implantation de l'interface forme sont proposées : par le type Carre et par le type Cercle.

Dans le main on voit ensuite qu'on peut comparer les périmètres de carres et de cercles, en utilisant les fonctions qui ont été définies à partir de l'interface Forme.

TODO Exercice 4.2.0 : Récupérez le fichier interface.go et testez-le.

TODO Exercice 4.2.1 : Implantez l'interface Forme une troisième fois par un type Rectangle qui représente des rectangles. Testez votre implantation.

TODO Exercice 4.2.2 : Sur le modèle de Perimetre, ajoutez une méthode Aire dans les méthodes demandées par l'interface Forme ainsi qu'une fonction PlusGrandeAire qui l'utilise. Mettre à jour les implantations de Carre, Cercle et Rectangle pour prendre en compte cette nouvelle méthode. Testez.

Jusqu'à présent nous avons vu comment créer des fonctions, avec un nom, des paramètres dont les types sont bien définis, et des valeurs de retour dont les types sont aussi bien définis. Nous avons aussi vu comment utiliser (appeler) ces fonctions dans du code Go pour les faire calculer des résultats. En Go (et dans un certain nombre de langages de programmation), il est possible de faire plus : passer les fonctions en paramètre d'autres fonctions,

retourner des fonctions en résultat d'autres fonctions. Ceci permet de créer simplement des fonctions modulaires, et donc de réutiliser le plus possible son code plutôt que de le dupliquer.

1.6.1.4.3. Fonctions anonymes

Avant de voir comment et pourquoi passer des fonctions en paramètres d'autres fonctions et retourner des fonctions en résultats d'autres fonctions, nous allons voir que l'on peut définir en Go des fonctions dites anonymes, c'est-à-dire des fonctions aux quelles on ne donne pas de nom.

Le programme anonyme.go montre deux exemples de déclarations de fonctions anonymes. On peut observer qu'une telle déclaration se fait exactement comme une déclaration de fonction classique, simplement sans spécifier de nom pour la fonction.

Exemple 1.47. anonyme.go

```
package main

import "log"

func main() {

 var x int = func(x, y int) int { return x + y }(2, 3)

 var f func(int, int) int = func(x, y int) int { return x - y }

 log.Print(x, f(2, 3))

}
```

La première fonction anonyme est immédiatement utilisée pour calculer une valeur (stockée dans la variable x). Pour cela, à la suite de la déclaration de la fonction on indique directement entre parenthèses les arguments avec les quels on souhaite l'appeler. Suite à ça la fonction «n'existe plus».

La deuxième fonction anonyme est stockée dans une variable f du bon type (donc du type de la fonction). Cela revient en fait exactement à avoir directement déclaré une fonction f comme on le fait d'habitude, et, concrètement, on utilisera rarement, voir jamais, cette syntaxe. Cependant, sur le même modèle, on pourrait avoir un type structuré dont l'un des champs est une fonction, qui peut donc être différente entre deux éléments de ce type (ce qui ne serait pas le cas pour une méthode s'appliquant aux éléments de ce type).

TODO Exercice 5.1.0 : Récupérez le fichier anonyme.go et testez-le. Assurez-vous d'avoir bien compris tout ce qui est fait dans ce code, et en particulier la façon dont on indique le type d'une (variable de type) fonction.E*n elles-mêmes, les fonctions anonymes ne sont pas très utiles (les exemples vus pour le moment consistent en une fonction qui n'est utilisée qu'une seule fois — elle pourrait être remplacée par son code — et une déclaration plus verbeuse que nécessaire d'une fonction classique), cependant on verra qu'elles nous servirons notamment lorsque l'on souhaitera passer des fonctions en paramètres d'autres fonctions.*

1.6.1.4.4. Fonctions en paramètres de fonctions

En Go, les fonctions peuvent prendre en paramètres d'autres fonctions. Ceci peut permettre de construire des fonctions plus générales que ce qu'on a fait jusqu'à présent. On peut en voir un exemple dans le paquet sort de la bibliothèque standard de Go. La fonction Slice 3 de ce paquet permet de trier un tableau selon n'importe quel ordre, cet ordre étant défini par une fonction.

Le programme tri.go trie le tableau t en ordre croissant. Pour cela, la fonction sort.Slice est appelée sur ce tableau avec comme deuxième paramètre une fonction qui indique que si t[i] est plus petit que t[j] alors il doit être avant dans le tableau trié.

Exemple 1.48. tri.go

```
package main

import (
```

```
 "log"
 "sort"
)

func main() {

 var t []int = []int{1, 3, 2, 0, -3, 27, 5}
 log.Print(t)

 sort.Slice(t, func(i, j int) bool { return t[i] < t[j] })
 log.Print(t)

}
```

TODO Exercice 5.2.0 : Récupérez le fichier tri.go et testez-le. Assurez-vous d'avoir bien compris tout ce qui est fait dans ce code, et en particulier la façon dont on utilise la fonction anonyme pour indiquer quel tri réaliser. Modifiez ce code pour que le tableau soit trié en ordre décroissant.*On pourrait aussi définir la fonction de comparaison d'éléments pour le tri de manière plus générale : elle pourrait comparer des éléments directement, et ne pas dépendre du tableau à trier. Cependant, pour des raisons techniques (pouvoir trier des tableaux de n'importe quel type d'éléments) c'est un autre choix qui a été fait par les développeurs du langage Go.*

Pour écrire soit même une fonction qui prend en paramètre une autre fonction, il suffit d'utiliser le mot clé func en définissant le type d'un paramètre.

Le programme applique.go déclare une fonction applique qui remplace chaque entier d'un tableau par le résultat d'une fonction f appliquée à cet entier. Ceci est ensuite utilisé pour doubler la valeur de tous les éléments d'un tableau.

Exemple 1.49. applique.go

```
package main

import "log"

func applique(t []int, f func(x int) int) {
 for i := 0; i < len(t); i++ {
  t[i] = f(t[i])
 }
}

func main() {

 var t []int = []int{1, 3, 2, 0, -3, 27, 5}
 log.Print(t)

 applique(t, func(x int) int { return 2 * x })
 log.Print(t)

}
```

TODO Exercice 5.2.1 : Récupérez le fichier applique.go et testez-le. Assurez-vous d'avoir bien compris tout ce qui est fait dans ce code.

TODO Exercice 5.2.2 : Écrivez un programme qui définit une fonction replie prenant en paramètre un tableau d'entiers t et une fonction f à deux arguments entiers et calculant f(f(. . .f(f(t[0],t[1]),t[2]) . . .),t[n-1]) puis qui utilise cette fonction pour calculer la somme des entiers contenus dans un tableau. On pourra supposer que les tableaux manipulés auront toujours au moins deux éléments.

1.6.1.4.5. Fonctions comme valeurs de retour de fonctions

Enfin, en Go, les fonctions peuvent retourner des fonctions. Ceci permet de créer des fonctions en fonction de paramètres, puis de les réutiliser.

Le programme tri2.go est similaire au programme tri.go. Cependant, au lieu d'utiliser une fonction anonyme pour indiquer comment trier le tableau, on génère cette fonction à l'aide d'une autre fonction : plusPetit. Ainsi, si on

souhaite trier un autre tableau de la même façon, on peut simplement appeler plusPetit sur ce nouveau tableau plutôt que de réécrire toute la fonction anonyme.

Exemple 1.50. tri2.go

```
package main

import (
 "log"
 "sort"
)

func plusPetit(t []int) func(i, j int) bool {
 return func(i, j int) bool { return t[i] < t[j] }
}

func main() {

 var t []int = []int{1, 3, 2, 0, -3, 27, 5}
 log.Print(t)

 sort.Slice(t, plusPetit(t))
 log.Print(t)

}
```

TODO Exercice 5.3.0 : Récupérez le fichier tri2.go et testez-le. Assurez-vous d'avoir bien compris tout ce qui est fait dans ce code. Modifiez le programme pour déclarer un deuxième tableau après t et triez ce deuxième tableau en utilisant la même méthode que pour trier t.*Ceci fonctionne car les tableaux sont basés sur des pointeurs (l'espace mémoire utilisé pour le contenu de t et le contenu de sa copie dans la fonction engendrée par plusPetit sont donc les mêmes) et car sort.Slice ne modifie pas la taille des tableaux (ce qui pourrait «séparer» les deux espaces mémoire, comme on l'a vu avec l'utilisation de append sur un tableau dont la capacité maximale était atteinte).*

TODO Exercice 5.3.1 Écrire une fonction add qui calcule et retourne la somme de deux entiers. Écrire une fonction buildSpecificAdder qui prend en paramètre un entier n et retourne une fonction qui prend en paramètre un entier, lui ajoute n et le retourne (par exemple, buildSpecificAdder(10) retournera une fonction à un argument qui ajoute 10 à son argument). La fonction buildSpecificAdder fournira donc une version spéciale de add dont l'un des arguments est fixé.

1.6.2. Travaux pratiques

Dans cette partie, vous aller progresser à votre rythme mais je vous invite fortement à venir faire vérifier votre code par votre enseignant pour ne pas prendre de mauvaise habitudes de programmation. Pour chaque exercice placez vous dans le répertoire te lancer les tests avec **go test**. Les tests sont des tests unitaires, ils reposent sur le paquetage de test intégré. Ce paquetage fournit tous les outils nécessaires à l'écriture et à l'exécution des tests.

Les tests fonctionne avec la convention de nommage pour tester `file.go` il faut un fichier `<file>_test.go` qui importe testing `import ("testing")`et qui contient des fonctions de test avec la convention Test<NomFonction>.

Les fonctions de test prennent en paramètre un pointeur sur la structure testing.T

```
func TestAdd(t *testing.T) {
// La logique de test est ici
}
```

testing fournie les fonctions pour remonter les erreurs : t.Error, t.Errorf, t.Log, t.Logf, ...

```
func TestAdd(t *testing.T) {
  result := Add(2, 3)
  expected := 5
  if result != expected {
    t.Errorf("Expected %d, got %d", expected, result)
  }
}
```

Des tests spécifique peut-être lancé avec **go test -run regexp** ou regexp est une expression régulière.

Important

Lors des évaluations tous les tests ne vous seront pas fournis. Il faut programmer pour répondre au problème et non pour passer les tests.

1.6.2.1. Recherche

Les exercices sont classés de manière arbitraire en 4 niveaux. Ici, tous les tests sont fournis, vous pouvez utiliser **go test**.

1.6.2.2. Récursivité

Ici go test permet de lancer tous les tests mais go test -run Etudiant permet de lancer des jeux de tests restreints, similaires à ceux que vous pourriez avoir en CC. Il faut coder pour répondre au problème et non pas pour passer les tests.

Important

A ne pas oublier, la (les) condition(s) d'arrêt(s), les appels récursifs qui convergent vers les conditions d'arrêt.

1.6.2.3. Pointeurs

En go les pointeurs sont "plus simples" qu'en C, par exemple, ils ne disposent pas d'une arithmétique. En go, les pointeurs sont des variables dont le contenu est une adresse mémoire d'une autre variable, c'est-à-dire l'adresse directe de l'emplacement mémoire d'une variable. Les pointeurs sont utilisés pour partager des données entre des fonctions, modifier des valeurs directement en mémoire et éviter la copie de grandes structures.

Pour rappels, l'opérateur * permet la déclaration et le déréférencement du pointeur et l'opérateur & permet d'accéder à une adresse.

```
package main

import "fmt"

func increment(ptr *int) {
    *ptr = *ptr + 1
}

func main() {
    var val int = 10
    var ptr *int = &val

    fmt.Println("adresse de val %p valeur de ptr %d \n", &val, ptr)
    fmt.Println("Avant l'appel :", val, *ptr)

    increment(&val)  // On passe l'adresse de val à la fonction
    fmt.Println("Après l'appel :", val, *ptr)
}
```

1.6.2.4. Structures

En go, une struct est une structure de données qui permet de regrouper plusieurs valeurs sous un même type : `https://gobyexample.com/structs`.

En go, une méthode est similaire à une fonction ordinaire, mais avec un récepteur (receiver) placé avant le nom de la fonction. Ce récepteur indique sur quel type cette méthode s’applique : `https://gobyexample.com/methods`.

1.6.2.5. Tri

TODO

1.6.2.6. Fichiers

TODO

Chapitre 2. SAÉ 1.01 : Implémentation d’un besoin client

Ce projet proposé et développé par Loïg Jezequel vise à mettre en place un système de particules. Il s’agit d’un outil fréquemment utilisé dans les jeux vidéos, les logiciels de 3D, les effets spéciaux pour le cinéma, etc. Un tel système permet de simuler graphiquement des effets complexes à représenter : explosions, feu, pluie, et bien d’autres. En général, les systèmes de particules sont utilisés en trois dimensions, cependant, pour simplifier, celui que vous allez développer travaillera en deux dimensions (principalement car l’affichage en trois dimensions serait assez complexe). La figure 1 montre deux exemples d’images que vous pourrez engendrer à l’aide de votre code (elles ont été produites à partir de ma version du projet).

Figure 2.1. Particules exemple 1

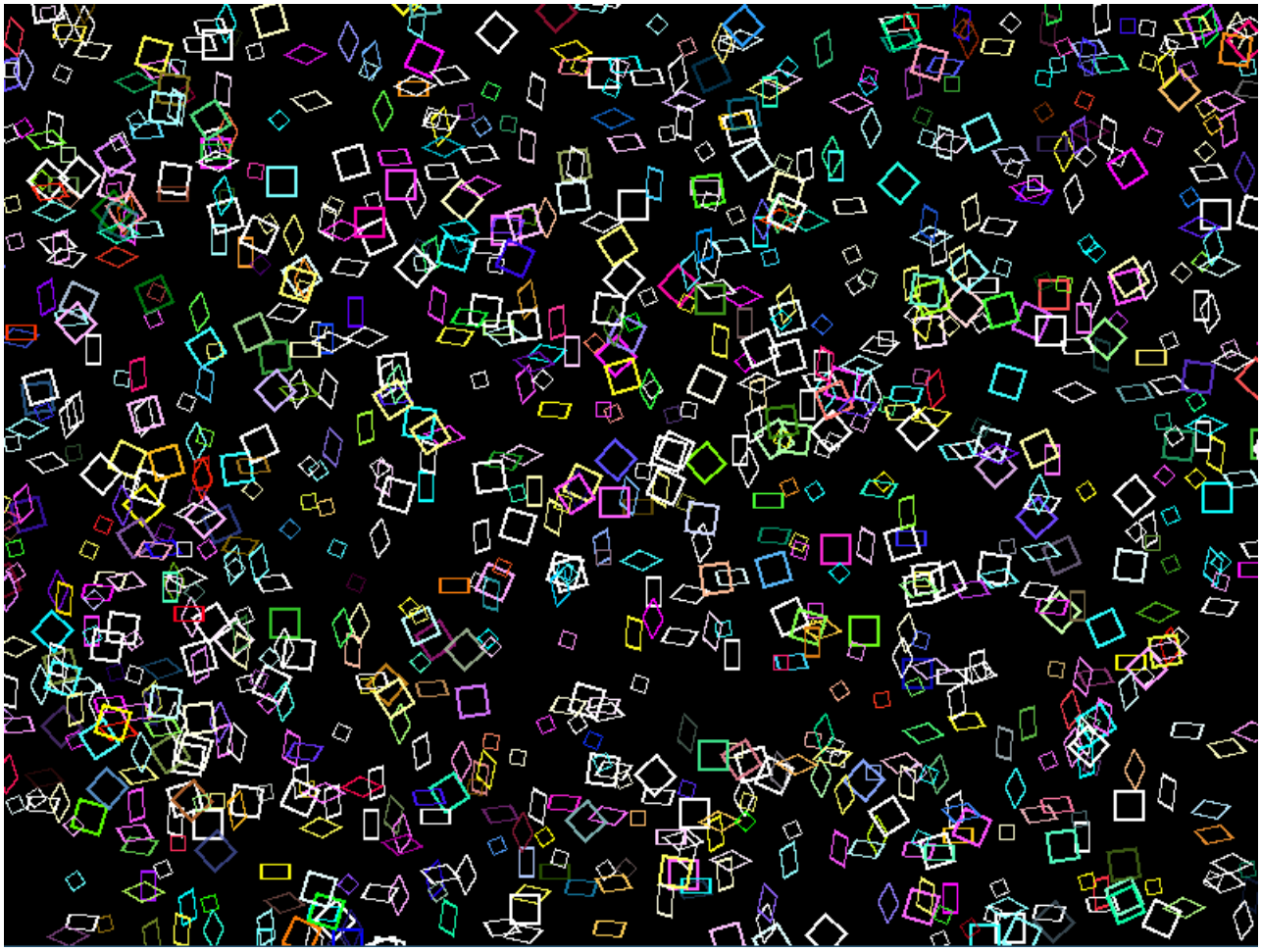
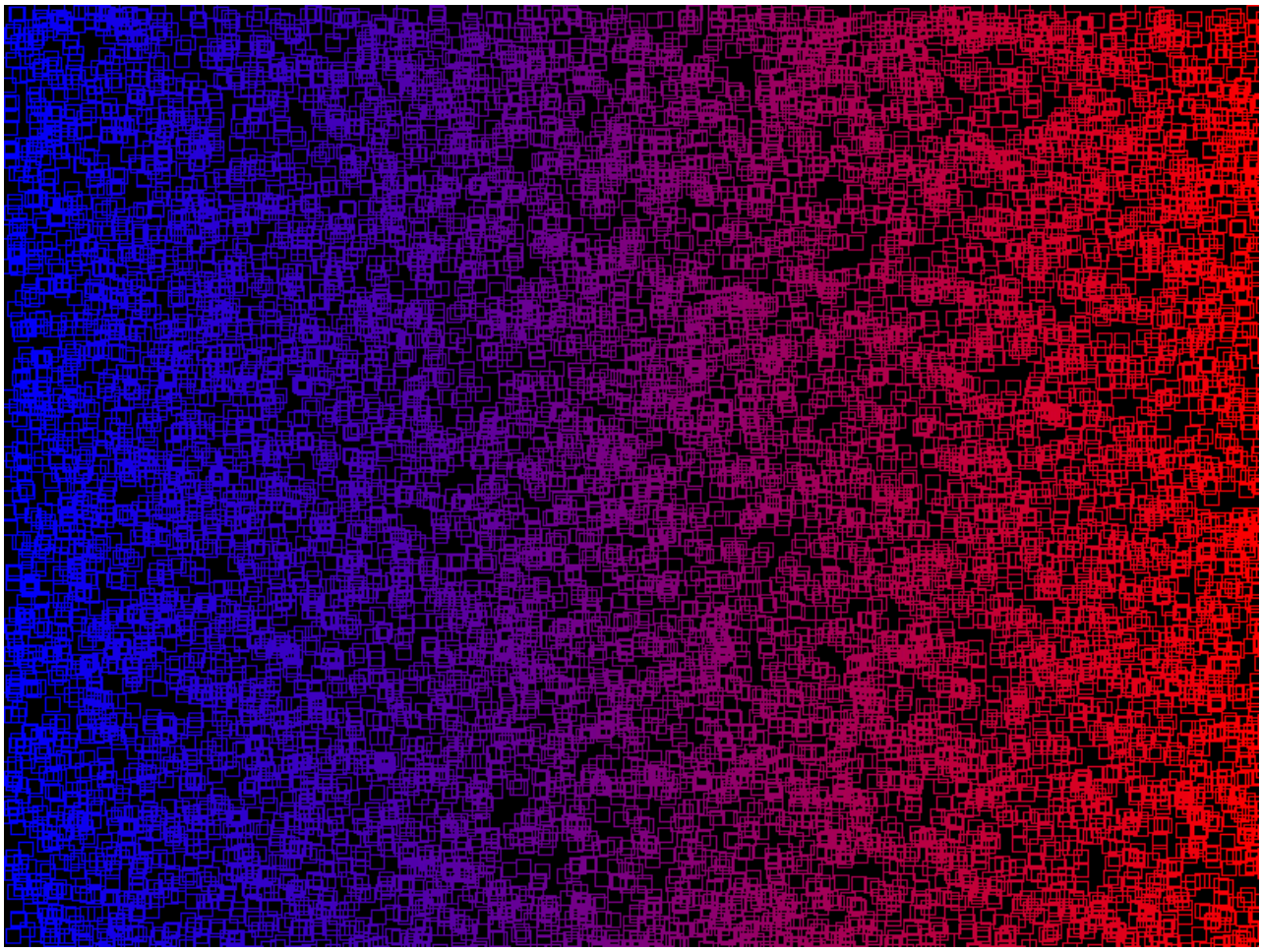


Figure 2.2. Particules exemple 2



Le premier exemple contient 1000 particules engendrées aléatoirement (couleur, position, taille en abscisse, taille en ordonnée, rotation), le second 10000 particules engendrées un peu moins aléatoirement : la couleur dépend de la position sur l'axe des abscisses.

1. Évaluation et dates

Ce projet participe à la SAE implémentation d'un besoin client, au sein de votre groupe de TP vous devrez constituer des équipes de 2, vous pouvez également être seul, une équipe de 3 est possible par groupe. Vous devez pendant cette séance constituer vos groupes ici : `https://uncloud.univ-nantes.fr/index.php/s/QHc6XcAcK4Wk7WQ`.

Vous aurez trois évaluations, une première en semaine 50, sur le système de base, vous devez avoir réalisé en snippets (sous git, dans code, choisir Snippets avant le vendredi 18h. La deuxième évaluation est en semaine 3 et 4, en semaine 3, vous devez 48 avant votre dernière séance de TP avoir réalisé un nouveau snippets, vous aurez passerez ensuite une revue de code avec un enseignant. Enfin, lors de la dernière séance de TP de la semaine 4, vous réaliserez une évaluation par les pairs.

Les attentes finales sont :

- Algorithmique
  - Un ou des problèmes algorithmiques, par exemple optimiser la détection de la collision entre particules, optimiser la gestion des particules qui quittent l'écran, …
  - Performance en temps et en espace
  - Lisibilité / documentation

- Programmation
  - Organisation du code
  - Visibilité, package
  - Tests
- Projets
  - Rendu
  - Sens
  - Usage

2. Principe général d’un système de particules

Un système de particules gère en ensemble de particules en mettant à jour leurs caractéristiques de manière régulière au cours du temps.

2.1. Particule

Une particule est un objet possédant :

- un certain nombre de caractéristiques permettant de l’afficher à l’écran : au minimum une position, mais éventuellement aussi un angle de rotation, une taille (ou une échelle par rapport à la taille d’une particule de base), une couleur, une transparence, etc.
- un certain nombre de caractéristiques permettant de faire évoluer son état au cours du temps : vitesse, accélération, durée de vie, fonctions d’évolution de la couleur ou de la transparence, capacité d’interaction avec les autres particules, etc.

2.2. Système de particules

Un système de particules gère un ensemble de particules en :

- engendrant de nouvelles particules au fil du temps en fonction de ses caractéristiques (taux de génération, position du générateur, etc),
- mettant à jour ses particules (en particulier leur position) au fil du temps en fonction des caractéristiques de celles-ci (en particulier leur vitesse),
- supprimant éventuellement les particules dont la durée de vie est épuisée.

3. Organisation des sources du projet

Des sources vous sont fournies avec le projet pour vous aider à implanter plus facilement votre système de particules. Vous devez respecter leur organisation tout au long du projet et vous ne devez pas la changer (si vous avez vraiment besoin de le faire ça doit être avec l’accord d’un enseignant et il faudra pouvoir le justifier).

3.1. Dossier principal

Le dossier principal du projet contient les fichiers go.mod et go.sum, le code de la fonction main (et des fonctions qu'elle utilise), un fichier config.json qui sert à configurer le projet, et les autres dossiers du projet.

3.1.1. Les fichiers go à la racine

Les fichiers de code go qu'on trouve dans ce dossier ne doivent pas être modifiés (à part éventuellement pour certaines extensions, quand c’est précisé). Ils se chargent de mettre à jour et d’afficher votre système de parti-

cules à un rythme régulier, soixante fois par seconde. Pour cela ils utilisent une bibliothèque tierce : `http://github.com/hajimehoshi/ebiten`.

Le programme peut fonctionner dans deux modes :

mode debug

> le nombre moyen d'images par seconde est affiché à l'écran, celui-ci devrait être d'environ 60, si ce n'est pas le cas c'est que votre code n'est pas assez efficace (ce mode est activé quand le champ Debug du fichier config.json vaut true),

mode normal

> le nombre moyen d'images par seconde n'est pas affiché (ce mode est activé quand le champ Debug du fichier config.json vaut false – ou si ce champ n'existe pas).

Durant tout le projet, si vous ajoutez des informations de debug (par exemple avec la bibliohèque log), laissez-les dans votre code en les mettant dans une conditionnelle qui permettra de les afficher seulement en mode debug (vous pouvez voir un exemple de comment faire cela dans le fichier Draw.go). Ceci permettra à la personne qui corrigera votre projet de voir plus en détails ce qui se passe si elle le souhaite, sans avoir à ajouter elle-même du code Go.

3.1.2. Fichier de configuration

Le fichier `config.json` permet de configurer le projet. Pour le moment il permet de définir la taille de la fenêtre d'affichage et son titre, de définir l'image utilisée pour représenter les particules, et de définir si on est en mode debug ou pas. Quelques autres paramètres sont aussi déjà disponibles dans ce fichier, qui seront utiles pour la première partie du projet mais qui n'ont pas d'effets particuliers pour le moment tant que vous n'avez pas écrit votre code.

3.2. Dossier assets

Le dossier `assets` contient les images utilisées pour le projet (une seule image en fait, le carré blanc représentant une particule) et les fonctions qui permettent de les charger en mémoire. Vous n'avez normalement jamais à toucher au contenu de ce dossier.

3.3. Dossier config

Le dossier `config` contient le code permettant de lire le fichier de configuration `config.json`. Vous pouvez éventuellement ajouter des choses ici si vous voulez enrichir ce fichier de configuration (ce qui peut permettre de tester différents paramètres de votre système de particules sans avoir à recompiler le programme à chaque fois : nombre de particules, point d'origine, gravité, etc).

3.4. Dossier particles

Le dossier particles contient le squelette du code que vous avez à écrire pour votre projet. Vous pouvez y ajouter autant de fichiers que vous voulez et vous pouvez ajouter du code dans les fichiers qui existent déjà. Cependant, il ne faut pas modifier les prototypes des fonctions existantes ni enlever des champs des types déjà définis (vous pouvez en ajouter par contre) car ils sont utilisés par les fonctions du main.

Fichier type.go

> Ce fichier définit les types de bases pour un système de particules : le système lui même (contenant une liste [1] de pointeurs vers des particules) et la particule (avec une position, une rotation, une échelle, une couleur et une transparence). Vous serez très probablement amenés à enrichir ces types.

Fichier new.go

> Ce fichier donne le prototype d'une fonction NewSystem() qui initialise un système de particules. Vous devrez écrire cette fonction.

[1] On utilise pour cela le paquet container/list sur lequel vous êtes très fortement invités à vous renseigner en allant lire sa documentation.

Fichier update.go

> Ce fichier donne le prototype d'une fonction Update() qui met à jour un système de particules. Cette fonction est appelée 60 fois par seconde par le main du programme principal sur un système de particules initialement créé par NewSystem(). Vous devrez écrire cette fonction. Dans toute la suite, quand on vous demande de modifier une fonction Update() c'est de celle-ci qu'il s'agit, à moins qu'on précise autre chose.

4. Premier travail : système de particules de base

Votre premier travail sera de coder un système de particules basique : chaque particule a une vitesse initiale (qui n'évolue pas au cours du temps), une position initiale, et sa position évolue au fil du temps en fonction de sa vitesse. Les autres caractéristiques des particules (taille, rotation, couleur, transparence) n'évoluent pas.

4.1. Fonction NewSystem()

Dans un premier temps il faudra coder la fonction NewSystem() (fichier particles/new.go) afin qu'elle crée un certain nombre de particules en leur donnant à chacune une position initiale. Les contraintes suivantes devront être respectées :

- le nombre de particules devra être InitNumParticles, qui est défini dans le fichier `config.json` (il faudra donc l'obtenir en lisant la variable General du paquet config),
- la position des particules devra être soit aléatoire soit la même pour toutes les particules, en fonction du champs RandomSpawn du fichier `config.json` (encore une fois, il faudra utiliser la variable `General` du paquet config pour obtenir cette valeur),
- si la position des particules est aléatoire il faudra utiliser le paquet math/rand pour la définir pour chaque particule, en s'assurant qu'elle reste toujours à l'intérieur de l'écran (la taille de l'écran se trouve dans la variable General du paquet config),
- si la position des particules n'est pas aléatoire elle devra être définie par les champs SpawnX et SpawnY du fichier `config.json` (encore une fois, il faudra utiliser la variable `General` du paquet config pour obtenir ces valeurs).

Les coordonnées à l'écran sont données en pixels à partir du coin supérieur gauche de la fenêtre : quand x augmente on se déplace vers la droite, quand y augmente on se déplace vers le bas.

Par défaut, une particule a une taille de 0, est transparente et n'a pas de couleur, elle n'est donc pas visible. Si vous voulez rendre vos particules visibles il faut changer ces trois caractéristiques.

La taille est définie par les champs ScaleX et ScaleY, s'ils valent tous les deux 1 la particule a sa taille de base (celle de l'image particle.png du dossier assets). Faire varier ScaleX (respectivement, ScaleY) modifie la taille de la particule selon l'axe des abs- cisses (respectivement, des ordonnées). La valeur indiqué dans ces champs multiplie la taille (avec 0.5 on divise la taille par deux, avec 2 on la multiplie par 2).

La transparence est définie par le champ Opacity. S'il vaut 1 la particule est complète- ment opaque et s'il vaut 0 elle est complètement transparente. Toutes les valeurs entre ces deux extrêmes sont possibles.

La couleur est définie par les champs ColorRed, ColorGreen et ColorBlue. Pour chacun d'entre eux, une valeur de 1 indique la présence de cette couleur à 100% et une valeur de 0 indique l'absence de cette couleur. Ainsi, si ces trois champs valent 0, la particule sera noire, et s'ils valent tous les trois 1, la particule sera blanche.

La fonction NewSystem() qui vous est fournie initialement affiche une particule au centre de l'écran.

4.2. Fonction Update() : vitesse des particules

Il faudra ensuite enrichir la fonction NewSystem() pour donner une vitesse initiale aux particules et coder la fonction Update() (fichier particles/update.go) pour les mettre à jour en fonction de cette vitesse : *C'est la fonction*

Update() du fichier update.go situé dans le répertoire particles qu'il faut modifier et surtout pas celle du fichier update.go situé à la racine du projet.

- Les vitesses seront définies selon deux coordonnées (x et y), on aura donc une vitesse pour chaque coordonnée,
- chaque particule aura ses propres vitesses, définies aléatoirement (utiliser math/rand) et une fois pour toutes par NewSystem(),
- la fonction Update() se contentera, pour chaque particule d'ajouter sa vitesse selon la coordonnée x à sa position en x et sa vitesse selon la coordonnée y à sa position en y,
- la fonction Update() est automatiquement appelée 60 fois par seconde, de manière régulière, par le programme principal (déjà codé) du projet.

4.3. Fonction Update() : génération de particules

Enfin, il faudra enrichir votre fonction Update() pour engendrer de nouvelles particules au cours du temps. Pour cela, on dispose d'un taux de génération (SpawnRate dans la config) qui indique combien de nouvelles particules doivent être engendrées à chaque appel à la fonction Update(). Il s'agit d'un nombre décimal (ainsi, s'il vaut 3 on doit engendrer 3 particules lors de chaque appel à Update() et s'il vaut 0.5 on doit engendrer une nouvelle particule lors d'un appel à Update() sur deux).

- Les nouvelles particules seront engendrées comme les particules de départ (position aléatoire ou la même pour toutes, selon la configuration, vitesse aléatoire),
- et elles seront bien sûr mises-à-jour comme les autres par Update() pour faire évoluer leur position au cours du temps.

Quand vous choisissez aléatoirement les vitesses des particules, vous pouvez plus ou moins contraindre leur choix. Ceci changera l'aspect de votre système de particules (par exemple des vitesses complètement aléatoires en x et y, aussi bien négatives que positives, avec toutes les particules engendrées au même point donnent un effet d'explosion). Faites des essais variés. Vous pouvez par exemple avoir plusieurs fonctions de génération des vitesses dans votre code, et permettre de sélectionner celle qu'on veut en ajoutant un champs pour cela au fichier de configuration.

5. Deuxième travail : extensions

La première partie du projet était très cadrée, cette deuxième partie est beaucoup plus libre. Vous devez coder quelques unes des extensions ci-dessous (elles sont à peu près classées par difficulté, de la plus simple à la plus compliquée, n'en faites pas que des simples, n'hésitez pas à interagir avec vos professeurs pour choisir vos extensions). Vous pouvez aussi proposer de nouvelles extensions, mais celles-ci doivent être validées par un professeur avant que vous commenciez à travailler dessus.

Chaque extension doit pouvoir être activée et désactivée en utilisant le fichier de configuration config.json (vous pouvez vous inspirer du fonctionnement du mode debug, activable dans config.json, pour cela). De même, si elles s'y prêtent, vos extensions doivent pouvoir être paramétrées dans le fichier config.json.

Vous rendrez votre travail avec un fichier de configuration config.json construit pour qu'en lançant votre projet sans rien toucher on obtienne un résultat à l'écran qui soit le plus impressionnant/joli possible. Vous pouvez aussi fournir d'autres fichiers de configuration (appelez-les config2.json, config3.json, etc) si vous souhaitez montrer différentes possibilités intéressantes de votre système de particules.

5.1. Gravité (Simple, pas de difficulté algorithmique)

Cette extension consiste à modifier vos particules pour prendre en compte la gravité. Pour cela, on va simplement utiliser une constante (l'accélération de la gravité),qui sera définie au niveau du système de particules, et prendra une valeur paramétrée par config.json. À chaque mise-à-jour d'une particule, cette valeur sera ajoutée à la vitesse selon la cordonnée y (l'ordonnée) de cette particule.

5.2. Extérieur de l’écran (Simple, pas de difficulté algorithmique)

Une particule qui est largement sortie de l’écran, de telle sorte qu’elle ne risque plus d’y revenir (ou en tout cas qu'on ne s’en rendra pas compte si elle ne revient pas), n’a pas de raison de continuer à être mise-à-jour et affichée. Pour détecter cela, on pourra utiliser un nouveau champs dans le fichier de configuration, indiquant la marge en pixels en dehors de l’écran en dehors de laquelle on arrête de mettre-à-jour et d’afficher une particule. Dans la fonction Update() on ne mettra à jour que les particules qui sont encore dans la zone définie par cette marge. De plus, on n’affichera aussi que ces particules. Pour cela, il faut modifier légèrement la fonction Draw() du fichier draw.go : il suffit d’ajouter une condition autour du code à l’intérieur de la boucle for.

5.3. Durée de vie (Simple, pas de difficulté algorithmique)

Il peut être intéressant, dans un système de particules, de donner une durée de vie aux particules. On peut ainsi représenter certains phénomènes plus facilement (par exemple, on peut représenter du feu en ayant des particules bougeant vers le haut et avec une durée de vie assez courte, les faisant disparaître — idéalement en devenant de plus en plus transparentes — au bout d’un certain temps). Cela signifie que, après un certain nombre d’appels à la fonction Update(), une particule arrête d’être mise-à-jour et d’être affichée. Pour mettre en oeuvre cela il faudra ajouter un champs dans le fichier de configuration, permettant de définir la durée de vie (en nombre d’appels à Update()) des particules. Chaque particule comptera combien de fois elle a été mise-à-jour et, si elle dépasse la durée de vie, ne sera plus mise-à-jour ni affichée (cette deuxième partie demande de modifier légèrement la fonction Draw() du fichier draw.go, il suffit d’ajouter une condition autour du code à l’intérieur de la boucle for).

5.4. Variations de couleur, d’échelle, de rotation, de transparence (Simple, pas de difficulté algorithmique, rendu visuel)

L’objectif d’un système de particules reste en général d’obtenir des résultats intéressants visuellement (même si on peut aussi les utiliser pour modéliser et observer des phénomènes physiques réalistes). Pour cela, il peut être pertinent de faire varier les caractéristiques visuelles des particules au cours du temps : couleur, taille, orientation, transparence.

En pratique, pour éviter de coder cela en dur dans la fonction Update() et pour permettre un peu de modularité, on peut associer, au système et/ou à chaque particule des fonctions chargées de faire évoluer ces différentes caractéristiques. Ces fonctions seront appelées sur chaque particule à chaque appel à Update().

Ceci est rendu possible en Go car une fonction est une valeur comme une autre, on peut donc avoir un champ d’un type structuré qui soit une fonction 2. On a alors la possibilité de définir un certain nombre de fonctions différentes de mise-à-jour et d’associer l’une d’entre-elles (à l’exécution) au système ou à chaque particule, en fonction de valeurs de champs du fichier de configuration.

Pour donner un exemple : on pourrait avoir une fonction increaseOpacity(p *Particle) qui augmenterait un peu l’opacité d’une particule à chaque fois qu’on l’appelle et une fonction decreaseOpacity(p *Particle) qui diminuerait un peu l’opacité d’une particule à chaque fois qu'on l’appelle. Dans le fichier de configuration on aurait un champ qui permettrait de choisir entre ces deux fonctions (un entier par exemple, 0 correspondant à aucune fonction, 1 à increaseOpacity(p *Particle) et 2 correspondant à de- creaseOpacity(p *Particle)). En fonction de cette valeur, on mettrait la fonction de gestion de l’opacité du système de particules soit à func(p *Particle), soit à increaseOpacity soit à decreaseOpacity. Et dans tous les cas, Update() appellerait la fonction de gestion de l’opacité du système sur chaque particule.

Cette partie est très vaste, il ne s’agit pas de tout faire, mais plutôt de proposer quelques fonctions de mise-à-jour des caractéristiques visuelles des particules qui vous paraissent intéressantes.

5.5. forme du générateur (Simple, pas de difficulté algorithmique, rendu visuel)

Pour l’instant, votre système de particules devrait engendrer des particules de deux manières différentes : aléatoirement positionnées à l’écran ou toutes en un même point. Il peut être intéressant de permettre d’engendrer des particules sur des formes plus complexes qu'un point. Par exemple, on pourrait les engendrer aléatoirement

réparties sur un cercle. Dans ce contexte il devient possible de choisir les vitesses initiales des particules en fonction de la forme du générateur. Par exemple, on pourrait faire que les particules sont initialement propulsées vers l'extérieur d'un cercle. À vous de voir comment mettre ça en oeuvre et comment le paramétrer dans le fichier de configuration. Cette partie est très vaste, il ne s'agit pas de tout faire, mais plutôt de proposer quelques formes de générateur et quelques choix de vitesses initiales associés qui vous paraissent donner des résultats intéressants.

5.6. Générateur mobile (Difficulté algorithmique moyenne, rendu visuel)

Le générateur de votre système de particules est immobile. Parfois on souhaiterait pouvoir le déplacer (imaginez par exemple qu'on l'utilise pour représenter du feu sur une torche portée par un personnage en mouvement) mais ceci crée des complexités : les particules doivent-elles suivre le générateur ? doivent-elles rester à leur place? Le déplacement du générateur influe-t-il sur la vitesse initiale des particules ?

Par ailleurs, ceci nécessite de modifier (légèrement) la fonction Update() principale du projet (pas celle que vous avez modifiée jusqu'à présent, mais celle qui agit sur un *game). C'est là qu'on va pouvoir gérer l'interaction avec le clavier et/ou la souris pour y réagir et modifier la position du générateur du système de particules. Pour savoir comment faire cela, il faudra consulter la documentation de la bibliothèque Ebiten : `https://pkg.go.dev/github.com/hajimehoshi/ebiten/v2`.

5.7. Modification dynamique du système de particules (Ergonomie, pas de difficulté algorithmique)

Même avec le fichier de configuration ce n'est pas toujours simple de trouver les bons paramètres pour avoir un résultat conforme à nos attentes à l'écran. On peut passer par un bon nombre d'essais où on modifie légèrement la configuration puis on teste à nouveau le programme. Il serait intéressant de pouvoir changer les paramètres du système à l'exécution, par exemple en utilisant le clavier pour sélectionner et modifier les différents paramètres. Ceci demande de développer une petite interface. Il faudra donc modifier la fonction Draw() pour afficher l'interface et la fonction Update() principale du projet (pas celle que vous avez modifiée jusqu'à présent, mais celle qui agit sur un *game) pour gérer l'interaction avec le clavier et/ou la souris. Pour savoir comment faire cela, il faudra consulter la documentation de la bibliothèque Ebiten : `https:// pkg.go.dev/github.com/hajimehoshi/ebiten/v2`.

5.8. Optimisation de la mémoire (Purement algorithmique, difficulté moyenne à difficile)

Cette partie nécessite d'avoir fait au moins l'extension 5.2 ou l'extension 5.3.

Une particule qui n'est plus mise-à-jour ni affichée (on dira qu'elle est morte) n'a pas besoin d'être gardée en mémoire. Si on laisse le système de particules engendrer de nouvelles particules pendant longtemps, la liste qui les contient ne cesse jamais de croître et contient de plus en plus de particules mortes. Pour éviter cela, on peut supprimer les particules mortes de cette liste : à chaque fois qu'une particule devient morte, on la retire de la liste.

5.9. Optimisation de la mémoire ++ (Purement algorithmique, difficile)

La méthode d'optimisation de la mémoire qui est présentée à l'extension 5.5 a quand même un défaut : les éléments retirés de la liste existeront toujours en mémoire, c'est simplement leur adresse qui aura été supprimée. Ceci pourrait finir par remplir tout l'espace disponible. Pour éviter cela, le (Runtime du) langage Go met en place un ramasse-miettes (garbage collector) qui va, de temps en temps, nettoyer la mémoire pour supprimer tous les éléments dans celle-ci qui ne sont plus référencés (pour lesquels il n'existe plus de variable qui contient leur adresse dans le programme) et donc en particulier nos particules mortes qui ne sont plus référencées dans la liste. Plus il y a de mémoire à nettoyer et plus cela prend du temps. De plus, pour des raisons de sécurité et d'intégrité des données, le fonctionnement normal du programme doit être interrompu pendant le nettoyage de la mémoire. On risque donc, au bout d'un moment, d'observer des ralentissements si on supprime ainsi les particules de la liste.

Pour résoudre ce problème on peut réutiliser les particules mortes ! Au lieu de supprimer des particules de la liste, on suivra la procédure suivante :

À chaque fois qu'une particule devient morte, on échange sa place avec la dernière particule vivante (non morte) de la liste.

À chaque fois qu'on doit engendrer une nouvelle particule on réutilise la première particule morte s’il y en a une, en la rendant vivante et en changeant les valeurs de ses différents champs : position, vitesse, etc (dans le cas contraire, on augmente, comme avant, la taille de la liste pour y mettre une particule nouvellement engendrée).

Pour être efficace, ceci demande de mémoriser à tout instant (dans un champs du type System l’élément contenant la première particule morte).

5.10. Collisions (Purement algorithmique, difficile)

Les particules dans votre systèmes n’ont pour l’instant aucune interactions entre elles. Il peut être intéressant de détecter quand deux particules se rencontrent (il y a une collision) et de réagir à ceci d’une manière ou d’une autre (en changeant la couleur, la taille, l’orientation, la vitesse, etc des particules qui se sont rencontrées). On peut même imaginer simuler les interactions physiques entre les particules : lors d’une rencontre elles se repoussent.